AI Infra 面试题库（合集）

AI Infra 面试题库 · 总览与使用说明

> 题库来源：原 `~/note/Interview/` 的 745 道题，经过逐题审查、去重、补缺之后，扩容为 **1109 道题**（其中包含 253 道思考与取舍题，以及 17 条追问链）。审查依据见 `../10-面试题库审查报告.md` 。带 ★ 前缀的题是面试必会题（P0）。2026-10-05 v3：15-项目考察一章对齐 04/05/06/07 的 v3 版本，题量从 56 题扩到 **150 题**（新增了微弹药 M1–M3、S6 调度插件、双主社区信任线，以及 ⚠ 自我纠错题）。2026-10-06 v3.1：**07/08/09/12 四章按照 AI Infra 平台岗的定位重新标注了层级**，增加了「P1 概念级 / P2 跳过」的分层说明，同时大幅压缩了 CUDA kernel 与算法侧题目的必会范围。

分层使用说明（v3.1 新增）

题库从过去的"全量均衡"升级为**按岗位分层**：

层级	含义	使用方式
★ = P0 必会	你能脱稿讲 60–120 秒，讲出来有框架、有例子，还经得起追问	第一轮只刷带 ★ 的题
P1 概念级	你能讲清楚"它是什么、会影响哪一项系统指标（显存/带宽/延迟/吞吐）"就可以	第二轮快速过一遍，能答出边界与代价即可
P2 跳过	这些是算法工程师和 CUDA kernel 工程师的主场，平台岗不考	第一轮直接跳过，但知识索引保留下来

分层标注只影响 07/08/09/12 四章，其余章节天然就贴合平台岗，不需要降档。

文件清单

文件	题数	思考题	定位
01-计算机基础	73	20	地基：CPU/缓存/内存/IO/网络/GPU 通路
02-Go语言	72	20	主力语言：GMP/GC/并发原语/工程实践
03-云原生	100	26	K8s 核心 + GPU 资源调度 + 弹性
04-分布式系统	72	20	一致性/Raft/事务/分片
05-存储内核	73	18	内核 I/O 栈 + 存储引擎

文件	题数	思考题	定位
06-网络内核	41	14	内核网络栈 + RDMA
07-数据流加速	105	20	P1 方向：DataLoader/缓存/预取/Checkpoint（★ 为核心，P2 为解码器）
08-推理引擎优化	133	30	P2 方向：vLLM/KV Cache/网关/PD 分离（★ 为核心，P2 为 kernel/量化）
09-ML 系统基础	60	14	与算法同学对话的桥（★ 为系统账，P2 为模型结构/微调算法）
10-性能工程	45	12	测量/定位/压测方法论
11-Python	45	10	DataLoader/数据管道
12-Cpp & CUDA	53	12	S2 支撑（★ 为 C/C++ 内存模型，P2 为 CUDA kernel 优化）
13-工程化与协作	30	8	测试/CI/开源流程
14-其他补充	57	12	可观测/SLO/FinOps/安全
15-项目考察	150	17（追问链）	**权重最高**：项目深挖加"防御的防御"（v3 已全量对齐新版项目）
合计	**1109**	**253**	

思考与取舍题（共 253 道）是什么

每个文件末尾都有一个 `## 思考与取舍题` 小节（15 号文件的 G 节"追问链"与它性质相同），共分五类题型：**反直觉追问、取舍辩论、假设破坏、量级估算、现象诊断**。这类题没有标准答案，它考察的是"你是不是真的理解了"——靠背题是过不了关的，你必须现场推理。它们也是区分"背题的人"和"真的做过的人"的分水岭，面试官最喜欢用这类题。

四轮使用法

1. **第一轮 · P0 过关**：只做全部带 ★ 的题。合格标准是你能口述 60–120 秒，讲出来有框架、有例子。答不出来的题记进错题本。
2. **第二轮 · 全量扫尾加错题回炉**：非 ★ 题快速过一遍，把错题本清零。
3. **第三轮 · 思考题自辩**：253 道思考题逐题**开口推理 2 分钟**（不许查资料），一旦推理卡壳，就说明这里是你的理解断点，回去重学对应的概念题。

4. **第四轮 · 模拟实战**：把 `15-项目考察` 脱稿长答（每题讲 5 分钟以上），再把各文件里的"综合设计题"拿来限时白板推演；找同学或 AI 扮演面试官和你对练，重点练习 G 节的追问链。

答案纪律

- **不写死答案，也不死背答案**：先自己作答，再用笔记、源码、论文校对，最后让 AI 扮演追问者来挑刺。
- 项目考察类题目的答案必须来自你真实的项目数据，禁止编造；其中 B/C 类题对应 `04/05-实施指南` 里的设计点，你每做完一个增量，就回来重答一次。
- 时间锚点：第一轮的 ★ 题要在 **2027 年 1 月底之前**完成（赶在暑期实习投递季之前）。

计算机基础面试题库（修订版）

面向 AI Infra 方向的计算机基础面试题库，覆盖体系结构、操作系统、存储、网络与 GPU 数据通路，共 73 题；★ 标记为面试必会的 P0 题。

CPU、缓存与 NUMA

1. 从 CPU 执行一条指令的完整流程出发，说明取指、译码、执行、访存、写回五个阶段中，哪些阶段可能成为 AI 训练任务的数据瓶颈。
2. ★ 什么是存储层次结构？寄存器、L1/L2/L3 Cache、主存、NVMe SSD、网络存储的延迟量级大致是多少？
3. ★ 解释 Cache Line 的概念，以及为什么 false sharing 会严重伤害多线程程序性能。
4. 什么是 NUMA？在 Kubernetes 节点上跑多 GPU 训练任务时，NUMA 绑定为什么重要？

中断、DMA 与 I/O 通路

5. 解释内存映射 I/O 与端口映射 I/O 的区别，以及它们在设备驱动中的典型应用。
6. 什么是 DMA？为什么高性能网络和存储设备几乎都依赖 DMA？
7. 解释中断、软中断、下半部机制，以及它们对高吞吐 I/O 场景的影响。
8. ★ 什么是零拷贝？列举 Linux 中常见的零拷贝技术，并说明各自适用场景。
9. 解释用户态与内核态切换的开销来源，以及系统调用为什么可能成为推理网关的瓶颈。

虚拟内存与内存管理

10. ★ 什么是虚拟内存？页表、TLB、缺页异常分别解决什么问题？
11. 解释大页内存的原理，以及它在 GPU 训练数据加载和 KV Cache 管理中的潜在价值。
12. 什么是内存屏障？在 Go 的并发模型和分布式存储复制协议中，内存屏障或等价语义为什么重要？
13. ★ 解释 buddy system 与 slab 分配器的原理：buddy 如何通过伙伴块的分裂与合并减少外部碎片？slab 如何在 buddy 之上减少内部碎片并缓存高频分配的对象？

14. 解释用户态内存申请路径：glibc malloc 什么时候通过 brk 扩展堆、什么时候通过 mmap 单独映射？两条路径与内核的交互方式以及内存回收行为有什么差异？

进程、线程与协程

15. 解释进程、线程、协程的区别，以及 Go runtime 调度器如何映射到操作系统线程。
16. 什么是 GMP 模型？Go 的调度器在高并发网络服务中相比传统线程模型有哪些优势与代价？

I/O 模型与事件驱动

17. 解释阻塞 I/O、非阻塞 I/O、I/O 多路复用、信号驱动 I/O、异步 I/O 的区别。
18. ★ 什么是 epoll？水平触发与边缘触发的区别是什么？在 Go netpoll 中如何体现？
19. 解释 Reactor 与 Proactor 模式，并说明它们在高并发推理网关中的适用性。
20. 什么是文件描述符？文件描述符耗尽时，系统层面通常有哪些表现和排查手段？
21. 解释 Linux 中一切皆文件的设计哲学，以及它对设备、管道、socket、cgroup 抽象的影响。

文件系统与存储 I/O

22. ★ 什么是 page cache？它如何影响训练数据读取、模型加载和日志写入性能？
23. ★ 解释 buffered I/O 与 direct I/O 的区别，以及 O_DIRECT 在数据库和分布式存储中的取舍。
24. 什么是 fsync、fdatasync？为什么分布式存储系统不能简单依赖 fsync 来保证一致性？
25. 解释 ext4、XFS、Btrfs 的基本设计差异，以及它们对大规模小文件训练数据集的影响。
26. 什么是 inode？inode 耗尽会导致什么问题？如何排查和缓解？
27. 解释硬链接与软链接的区别，以及它们在容器镜像层中的表现形式。
28. 什么是 VFS？它如何统一不同文件系统的接口？
29. 解释 FUSE 的原理，以及它为什么可能成为训练数据流加速中的性能瓶颈。
30. ★ 什么是 io_uring？它相比 epoll + 传统 AIO 有哪些优势？适合哪些 AI Infra 场景？

并发与同步

31. 解释 CPU 缓存一致性协议 MESI，以及多核环境下共享数据结构的性能陷阱。
32. 什么是伪共享？如何在 Go 中通过 padding 或数据结构布局缓解？
33. 解释原子操作与锁的区别，以及 CAS 在无锁队列和分布式协调中的应用。
34. 什么是自旋锁、互斥锁、读写锁？分别适合什么竞争强度和临界区长度？
35. 解释条件变量与 channel 的语义差异，以及 Go 中何时该用 channel、何时该用 mutex。
36. 什么是死锁、活锁、饥饿？在分布式训练任务调度中如何避免？

网络协议与低延迟传输

37. ★ 解释 TCP 三次握手与四次挥手，以及 TIME_WAIT 过多对推理网关的影响。
38. 什么是 TCP 粘包与拆包？在自定义 RPC 协议中如何设计消息边界？
39. 解释 Nagle 算法与 TCP_NODELAY，以及它们对低延迟推理请求的影响。
40. 什么是拥塞控制？CUBIC、BBR 的基本思想差异是什么？对跨机房模型传输有什么影响？
41. 解释 HTTP/1.1、HTTP/2、HTTP/3 的关键差异，以及 gRPC 为什么常基于 HTTP/2。
42. 什么是 TLS 握手？会话复用、0-RTT、mTLS 在服务网格和推理网关中的作用是什么？

GPU 与高性能数据通路

43. ★ 解释 RDMA 的基本原理，以及它在分布式训练 AllReduce、参数同步和存储访问中的价值。
44. 什么是 RoCE 与 InfiniBand？它们相比 TCP 在 AI 集群中的优势和运维挑战是什么？
45. ★ 解释 GPU 与 CPU 之间的数据通路：PCIe、NVLink、NVSwitch 的带宽和拓扑差异。
46. 什么是 pinned memory？为什么 GPU 训练中的数据加载常使用 pinned memory？
47. 解释 CUDA Stream 与异步拷贝，以及它们如何影响数据加载与计算重叠。
48. ★ 什么是 GPUDirect Storage？它试图解决什么问题？对训练数据流加速意味着什么？
49. 解释 SSD 的读写放大、垃圾回收、TRIM，以及它们对高吞吐数据管道的影响。
50. 什么是 RDMA over Converged Ethernet 的 PFC 与 ECN？为什么无损网络对 AI 集群重要？

容器隔离与资源控制

51. 解释 cgroup 与 namespace 的区别，以及它们如何共同构成容器隔离的基础。
52. 解释 cgroup v1 与 v2 的主要差异：统一层级、no-internal-process 约束、控制器委托与 PSI 压力反馈，对容器运行时和 Kubernetes 资源管理有什么影响？

思考与取舍题

54. 用 O_DIRECT 做备份与恢复，速度反而比 buffered I/O 更慢——列出三个候选原因并按可能性排序。
55. 估算 8 卡 H100 节点上 NVLink 总带宽与 PCIe 总带宽的量级差，这个差值如何决定 AllReduce 算法与跨机通信方案的选择？
56. 把 L3 缓存扩大 4 倍但访问延迟翻倍，对训练吞吐未必是好事——什么样的负载特征会让这笔交易亏损？
57. NUMA 绑定能降延迟，但把训练进程的内存严格绑死在单个节点上，什么场景下反而酿成事故？
58. 大页能减少 TLB miss，但请各构造一个它让缺页处理变慢、让内存碎片恶化的反例。
59. epoll 边缘触发相比水平触发省下的系统调用开销最终去了哪里？什么条件下这点节省会被读写循环的复杂度反噬？
60. 中断风暴与轮询空转分别出现在什么负载区间？两者的临界切换点由哪些因素决定？
61. 把掉电窗口从秒级压到接近零，吞吐与尾延迟会先在哪里崩掉？组提交在这笔交易里扮演什么角色？

62. 零拷贝在 TLS 网关里往往名存实亡——哪些中间层会把"零拷贝"悄悄变回两次拷贝？
63. pinned memory 让主机到显卡的拷贝更快，为什么它用多了反而可能拖累整机与其他进程？
64. 量级估算：每秒 10 万请求、每请求 20 次系统调用的推理网关，系统调用与上下文切换的开销占 CPU 的比例有多大？
65. 训练节点内存带宽已经打满，CPU 利用率却只有 30%——列三个候选原因并按可能性排序。
66. 估算把 100GB 的 checkpoint 从 NVMe 装进 8 卡 H100 的最短加载时间，瓶颈卡在哪一段链路上？
67. 把网卡中断绑定到与 GPU 同 NUMA 的核上，为什么有时反而让那个核成为瓶颈？该怎么判断该绑还是该分散？
68. 假设把 page cache 从内核里删掉，哪些系统会最先失败？分布式存储客户端为什么反而要自建缓存？
69. CPU 频率显示正常、机器负载很低，训练性能却只有标称的一半——列三个候选原因并按可能性排序。
70. 取舍辩论：跨机训练流量走 RDMA 还是 TCP？从运维代价、故障域与带宽利用率三个角度，TCP 阵营最硬的牌是什么？
71. 量级估算：一张 7GB/s 的 NVMe、一张 25GB/s 的网卡、一条 32GB/s 的 PCIe 通道，训练数据加载链路的短板在哪一块？请把三者换算到同一单位。
72. 假设把 buddy system 换成只分配固定尺寸块的分配器，哪些负载受益、哪些立刻失败？
73. 节点 free 显示还有几百 GB available，新进程却触发 OOM——列三个候选原因并按可能性排序。

综合设计题

53. ★ 从一次推理请求进入网关开始，到 GPU 返回 token，列出沿途可能涉及的操作系统、网络、存储、GPU 数据通路关键环节。

Go 语言面试题库（修订版）

面向 AI Infra 方向的 Go 语言面试题库，覆盖调度器、内存与 GC、并发原语以及高并发网关工程实践，共 72 题；★ 标记为面试必会的 P0 题。

GMP 与调度器

★1. 解释 Go 的 GMP 调度模型，G、M、P 各自承担什么职责。 2. 为什么 Go 需要 P？没有 P 的 M 和没有 M 的 G 分别处于什么状态？ ★3. 解释 Go 调度器中的 work stealing 机制，以及它如何提升多核利用率。 4. 什么是 handoff？M 阻塞在系统调用时，P 和 G 会发生什么？ 5. 解释 Go 的抢占式调度，协作式抢占与异步抢占的差异是什么。 ★6. 什么是 goroutine 泄漏？在高并发推理网关中如何排查和避免？ ★7. 解释 Go netpoll 与 epoll 的关系，以及网络 I/O 就绪后如何唤醒 G。 8. 为什么 Go 能用同步阻塞的代码风格写出高并发网络服务？ 9. 解释 runtime.Gosched、runtime.LockOSThread 的使用场景与代价。 10. 什么是 GOMAXPROCS？它如何影响网关吞吐与 GPU 数据加载进程？

内存分配与垃圾回收

11. 解释 Go 的内存分配器：mcache、mcentral、mheap 的层级结构。

12. 什么是 size class？Go 为什么按对象大小分级分配？ ★13. 解释逃逸分析，以及哪些常见写法会让变量逃逸到堆上。
13. 什么是栈增长？Go 的 goroutine 栈如何动态扩缩容？ ★15. 解释 Go GC 的三色标记法，白色、灰色、黑色分别代表什么。 ★16. 什么是写屏障？混合写屏障解决了什么问题？
14. 解释 Go GC 的 STW 阶段，以及为什么现代 Go 的 STW 已经非常短。
15. 什么是 GOGC？如何通过 GOGC 和 GOMEMLIMIT 调优推理网关的 GC 行为。
16. 解释 GC assisted，为什么分配快的 goroutine 会被拉去帮忙做 GC。
17. 什么是内存归还？Go 什么时候把内存还给操作系统？
18. 解释 sync.Pool 的原理，以及它在网关请求对象复用中的价值与陷阱。
19. 什么是对象复用？如何避免 sync.Pool 中的对象被 GC 误回收。

channel、context 与同步原语

★23. 解释 Go 的 channel 底层结构，发送、接收、关闭分别做了什么。 24. 有缓冲 channel 与无缓冲 channel 的语义差异是什么？ 25. 向已关闭的 channel 发送数据、重复关闭 channel 分别会发生什么？ 26. 解释 select 的随机性，以及如何用 select 实现超时与取消。 ★27. 什么是 context？context 的取消传播在推理网关中如何工作？ 28. 解释 context.WithTimeout、WithCancel、WithValue 的使用边界与陷阱。 29. 解释 sync.Mutex 的正常模式与饥饿模式。 30. RWMutex 适合什么场景？为什么它不一定比 Mutex 快？ 31. 解释 atomic 包提供的内存序语义，以及它在无锁结构中的应用。 32. 什么是 CAS？如何用 CAS 实现无锁计数器或队列？ ★33. 解释 Go 的内存模型，happens-before 关系如何保证并发可见性。 34. 什么是 data race？go test -race 能发现哪些问题、漏掉哪些问题？ 35. 解释 WaitGroup 的正确用法，以及常见的 Add 与 Wait 竞态错误。 36. 什么是 errgroup？它如何简化并发任务的错误传播与取消。 ★37. 解释 singleflight 的原理，以及它在模型权重加载、缓存回源中的应用。 38. 解释 sync.Once 与 sync.Map 的适用场景，以及 sync.Map 的性能陷阱。

网关工程实践

★39. 什么是限流器？如何用 token bucket 在 Go 网关中实现平滑限流。 ★40. 解释 Go 中的背压，如何用有缓冲 channel 和信号量控制并发。 ★41. 什么是优雅关闭？如何让 HTTP server 在滚动更新时不丢请求。 42. 解释 http.Server 的 Shutdown 流程，以及它和长连接、流式响应的关系。 43. 什么是连接池？Go 的 http.Transport 如何管理长连接与空闲连接。

可观测性与语言底层

★44. 在 SSE 长连接推理网关中，goroutine 泄漏的典型场景有哪些？如何用 pprof 的 goroutine profile 定位泄漏点与阻塞调用栈？ ★45. 解释 Go 的 pprof：CPU、heap、block、mutex、goroutine profile 分别怎么看。 46. 什么是 trace？如何用 go tool trace 分析调度延迟与 GC 抖动。 47. 解释 benchmark 的写法，以及如何避免编译器优化掉被测代码。 48. 什么是内存逃逸与 false sharing 在 Go 中的表现，如何用 padding 缓解？ 49. 解释 Go 的 unsafe.Pointer 与 uintptr 的区别，以及使用 unsafe 的风险。 ★50. 解释 Go 泛型的实现

方式（GC shape stenciling）：为什么泛型没有对所有类型做完全的单态化？接口装箱又会对热路径性能造成哪些影响？

Kubernetes 控制器中的并发

51. 在 K8s 控制器中，Informer 与 WorkQueue 如何利用 Go 并发原语协作？

思考与取舍题

53. Go 的 GC 为什么始终不做分代？如果强行引入分代假设，Go 的哪种常见写法会第一个打破它？
54. 假设把 GC 改成"平时极快、每 10 轮 STW 一次 100ms"，推理网关最先出现什么症状？哪条监控曲线先亮？
55. 构造一个 channel 反而比 mutex 慢一个量级的场景，并指出慢在哪条路径上。
56. 逃逸分析把热路径的堆分配全部消除后，GC 压力并不会凭空消失而是发生"转移"——转到了哪里？什么现象会让你误判优化无效？
57. sync.Pool 的对象可能在两轮 GC 后被清空：什么负载下命中率会断崖式下跌？把它包成"永不过期"又会引入什么新问题？
58. goroutine 数量随流量涨落是正常水位，泄漏则是单调缓涨——如何设计告警规则把"高并发"与"慢泄漏"区分开，避免误报与漏报？
59. runtime.LockOSThread 会把 G 绑死在一个 M 上：当 cgroup 只给 2 个 CPU 配额而宿主机有 64 核时，这个绑定会怎样与配额互动出问题？
60. GOMAXPROCS 在容器里默认取宿主机核数，这个默认值在什么条件下会错？设大了与设小了分别会观察到什么症状？
61. benchmark 跑分漂亮、上线后 P99 难看——至少列出三类基准测试与真实负载之间的偏差来源。
62. 取舍辩论："context 必须穿透所有层"与"热路径上 WithTimeout 的分配开销不可接受"，双方各自的道理是什么？
63. 假设删掉 netpoll，让每条连接独占一个系统线程，1 万条 SSE 长连接的网关会以什么方式败掉？
64. 量级估算：10 万连接、每连接 3 个 goroutine 的网关，仅 goroutine 栈与运行时元数据的内存量级是多少？什么时候会突破 10GB？
65. GC 结束后 heap 的 inuse 明显下降，进程 RSS 却居高不下——列三个候选原因并按可能性排序。
66. 把 GOGC 从 100 调到 400 通常省 CPU 但费内存，什么场景下会两头都变差？
67. GOMEMLIMIT 逼近时 GC assist 制造延迟尖刺，超限又可能被 OOM kill——推理网关该把这条线画在哪里？依据是什么？
68. 同一份代码，物理机上 P99 平稳，进容器后每两分钟出现一次 50ms 尖刺——列三个候选原因并按可能性排序。
69. 把 select 对多个就绪 case 的伪随机选择改成固定优先级，会让哪类程序从"正确"悄悄变成"饥饿"？
70. 互斥锁饥饿模式的触发阈值建立在什么假设之上？推理网关的哪种负载会打破它？
71. 量级估算：在 netpoll 路径上写出一次 4KB 响应，涉及的内存拷贝次数与上下文切换次数各是多少量级？
72. 假设 Go 的内建 map 从语言层面变成并发安全，哪些现有程序反而会变慢？这份代价被转移到了哪里？

综合设计题

★52. 如果让你用 Go 写一个 vLLM 前面的高并发推理网关，你会如何设计 goroutine 模型、连接管理、限流、超时、背压与优雅关闭?

云原生面试题库（修订版）

面向 AI Infra 方向的云原生面试题库，覆盖容器与运行时、Kubernetes 核心、GPU 资源与调度、集群批调度与弹性扩缩，共 100 题；★ 标记为面试必会的 P0 题。

容器与运行时

1. ★ 从容器本质出发，解释 namespace 与 cgroup 分别解决了隔离和资源限制中的什么问题。
2. ★ 为什么容器不是轻量级虚拟机？容器共享内核带来的性能优势和安全代价分别是什么?
3. ★ 解释 runc、containerd、CRI-O、Docker 之间的关系，以及 Kubernetes 为什么最终通过 CRI 解耦运行时。
4. 什么是 OCI 镜像规范与 OCI 运行时规范？它们分别约束了哪一层?
5. 解释镜像分层与联合文件系统的原理，以及为什么镜像层设计会影响模型加载速度。
6. 什么是 Copy-on-Write？它在容器启动、模型文件读取和写时复制场景中的性能特征是什么?
7. 解释 cgroup v2 在 kubelet 中的落地差异：统一层级、cpu.weight 与 memory.qos 等接口变化，以及 cgroupfs 与 systemd 两种驱动对资源管理、观测与 eBPF 生态有什么影响?

Pod 与工作负载

8. ★ 解释 Kubernetes Pod 的设计哲学，以及为什么 Pod 是原子调度单位而不是容器。
9. 什么是 Init Container、Sidecar、Ambassador、Adapter 模式？分别适合什么 AI Infra 场景?
10. 解释 Pod 的生命周期：Pending、Running、Succeeded、Failed、Unknown，各阶段可能卡在哪些系统层面。
11. 什么是 Pause 容器？它在 Pod 网络与 PID namespace 共享中起什么作用?
12. ★ 解释 Deployment、StatefulSet、DaemonSet、Job、CronJob 的语义差异与适用场景。
13. 为什么有状态训练任务、分布式推理服务、日志采集组件通常选择不同的工作负载控制器?

网络与服务暴露

14. ★ 解释 Service、Endpoints、EndpointSlice 的关系，以及大规模推理服务下 EndpointSlice 为什么重要。
15. 什么是 ClusterIP、NodePort、LoadBalancer、ExternalName？各自适合什么流量入口场景?
16. ★ 解释 kube-proxy 的 iptables 模式与 IPVS 模式差异，以及高并发推理网关下如何选型。
17. 什么是 Ingress 与 Gateway API？它们和 Service 的边界在哪里?

18. 解释 NetworkPolicy 的实现原理，以及为什么它依赖 CNI 插件的能力。
19. 什么是 CNI？Calico、Cilium、Flannel 的基本差异是什么？
20. 解释 Overlay 网络与 Underlay 网络的区别，以及它们对 AI 集群东西向流量的影响。
21. ★ 什么是 eBPF？Cilium 如何利用 eBPF 做网络、可观测性和安全策略？

调度与资源管理

22. ★ 解释 Kubernetes 调度框架的扩展点：Filter、Score、Reserve、Permit、Bind 等。
23. ★ 结合调度器扩展实战，解释 Reserve 与 Permit 扩展点的语义，以及 PodSchedulingGate 如何实现任务的门控与批量放行？在 GPU 集群中如何组合三者实现"先等待、再锁定资源、最后绑定"的调度流程？
24. ★ 什么是 Request 与 Limit？CPU 和内存的 Request/Limit 在调度与运行时分别如何生效？
25. 解释 QoS 类别 Guaranteed、Burstable、BestEffort，以及它们对驱逐顺序的影响。
26. 什么是 Eviction？节点资源压力下，kubelet 如何决定驱逐哪些 Pod？
27. 解释拓扑管理器与 CPU Manager，它们如何影响低延迟推理和 GPU 训练任务？

GPU 资源与调度

28. ★ 什么是 Device Plugin？GPU 厂商如何通过它把 GPU 资源暴露给 Kubernetes？
29. 解释 NVIDIA GPU Operator 的作用，以及它管理的组件大致有哪些。
30. ★ 什么是 MIG？它和整卡分配、时间片共享相比，在推理服务隔离性上有什么取舍？
31. 解释 vGPU 与 MIG 的差异：两者在虚拟化实现方式（驱动层拦截与调度 vs 硬件划分）、隔离边界、故障域、许可成本与典型场景上分别有什么不同？
32. 什么是 GPU 算力切分与显存切分？只切分显存而不限制算力占用时，多个租户之间会互相造成什么影响？
33. 解释 GPU 资源超卖的风险：时间片轮转带来的上下文切换开销与延迟抖动在什么条件下出现？为什么在线推理与离线训练应当采用不同的超卖策略？
34. 解释 NVIDIA MPS 的原理：它如何让多个进程共享同一 GPU 的 SM？相比时间片共享有什么优势？适合什么场景，又会引入哪些隔离与故障域问题？
35. 什么是 GPU 拓扑感知调度？NVLink、PCIe、NUMA 亲和性分别如何影响多卡训练与推理的通信性能？调度器应如何利用拓扑信息打分？
36. 解释 NUMA 感知调度在 Kubernetes 中的现状与挑战，尤其是多 GPU 多网卡场景。
37. 什么是 RuntimeClass？它如何支持 gVisor、Kata Containers、NVIDIA 容器运行时等不同运行时？
38. MIG 设备在 Kubernetes 中如何表示为扩展资源？一块 GPU 切分出的 MIG 实例如何通过 nvidia.com/mig-1g.5gb 这类资源名完成上报与分配？
39. ★ 解释 DRA（Dynamic Resource Allocation）与 NRI（Node Resource Interface）分别解决什么问题？相比 Device Plugin，它们在资源建模、分配灵活性与节点设备可观测性上补齐了哪些能力？
40. 如何基于 scheduling framework 编写自定义 GPU 打分插件？你会依据哪些信号（碎片率、拓扑亲和、剩余显存、任务类型）设计 Score 逻辑，又如何避免打分插件拖慢调度吞吐？

41. 什么是调度器 Profile？如何为在线推理与离线训练配置不同的过滤、打分与抢占策略？

存储与状态管理

42. ★ 解释 Kubernetes 中存储的核心抽象：Volume、PersistentVolume、PersistentVolumeClaim、StorageClass。
43. ★ 什么是 CSI？它如何解耦存储供应商与 Kubernetes 核心代码？
44. 解释 ReadWriteOnce、ReadOnlyMany、ReadWriteMany、ReadWriteOncePod 的语义差异。
45. 什么是动态卷供给？Provisioner、Attacher、Detacher、Snapshotter 各自负责什么？
46. 解释 StatefulSet 中 volumeClaimTemplates 的作用，以及有状态训练任务如何持久化检查点。
47. 什么是 Local PV？它在高吞吐训练数据缓存和临时检查点场景中的优势与风险是什么？

配置、身份与准入

48. 解释 ConfigMap 与 Secret 的差异，以及 Secret 在 etcd 中的存储与加密问题。
49. 什么是 ServiceAccount？它在 Pod 访问 API Server 和云上存储时的作用是什么？
50. 解释 RBAC 的核心模型：Role、ClusterRole、RoleBinding、ClusterRoleBinding。
51. ★ 什么是 Admission Controller？Validating 与 Mutating 的先后顺序是什么？

控制面与控制器模式

52. ★ 解释 Kubernetes Operator 模式，以及为什么分布式训练框架常用 Operator 管理。
53. ★ 什么是 CRD？它和自定义控制器如何共同实现声明式 API？
54. ★ 解释 Informer、InformerFactory、Lister、WorkQueue 在 controller 中的协作机制。
55. 解释 CRD status 大对象与 etcd 单条记录约 1.5MB 限制的关系：大规模集群下 informer cache 的内存压力来自哪里？有哪些缓解手段？
56. ★ 什么是 Leader Election？为什么高可用控制器必须处理脑裂与租约续期？
57. 解释 Kubernetes 的 watch 机制与 resourceVersion，以及大规模集群下 watch 的压力来源。
58. 什么是 APF？它如何保护 API Server 不被突发控制器流量打垮？
59. ★ 解释 etcd 在 Kubernetes 中的角色，以及 etcd 性能恶化时集群会出现哪些典型症状。
60. 什么是 Pod Disruption Budget？它在滚动更新、节点维护和训练任务抢占中如何起作用？

弹性扩缩容

61. ★ 解释 HPA、VPA、Cluster Autoscaler、Karpenter 的差异，以及推理服务弹性扩缩容的难点。
62. 解释 Cluster Autoscaler 如何扩容 GPU 节点，以及 Karpenter 相比它的优势：两者在节点供给模型、扩容速度与 bin-packing 能力上的差异，对 GPU 集群选型意味着什么？
63. 什么是 KEDA？它如何基于队列深度等外部指标做事件驱动扩缩容？相比 HPA 自定义指标的方式简化了什么？

64. 解释推理服务弹性扩容的冷启动问题：镜像拉取与模型权重加载为什么慢？模型预热与就绪探针如何让新副本快速进入可服务状态？

集群批调度

65. 什么是 Kueue？它如何通过 ClusterQueue 与 LocalQueue 实现批任务的排队与配额管理？workload 从 Pending 到被 Admit 经历哪些状态？
66. 解释 Volcano 在 AI 训练与推理调度中的角色：它提供了哪些 kube-scheduler 默认不具备的能力？
67. 什么是 Gang Scheduling？为什么分布式训练必须以 all-or-nothing 方式调度一组 Pod？部分调度会带来什么资源浪费与死锁问题？
68. 解释 Coscheduling 与 Gang Scheduling 的实现差异：两者在调度插件机制、等待超时与优先级处理上的取舍是什么？
69. 什么是 Bin Packing？它如何通过装箱打分提升 GPU 利用率？过度追求装箱又可能伤害什么？
70. 解释 GPU 集群中的碎片问题：小卡任务散布如何导致大卡任务无法调度？有哪些缓解碎片、提升大模型训练任务调度成功率的实践？
71. 什么是资源预留与抢占？如何保证高优先级训练或推理任务在资源紧张时拿到 GPU？抢占的代价与风险是什么？
72. 解释 PriorityClass 在 Kubernetes 中的作用：优先级如何影响调度顺序与抢占行为？系统级关键优先级为什么不能滥用？
73. 什么是 Descheduler？它基于哪些策略腾挪 Pod？为什么它能间接提升 GPU 集群的装箱率与负载均衡水平？

思考与取舍题

75. 假设删掉 EndpointSlice（连同背后的 endpoint 分发机制），一个 5000 副本、频繁发布的推理服务会以什么方式退化？控制面与数据面分别先坏在哪里？
76. Device Plugin 为什么被认为终将被 DRA 替代？这场迁移的成本主要压在谁的身上？
77. IPVS 并不总在大规模下赢 iptables——构造一个让 IPVS 反而劣化的条件组合。
78. PodSchedulingGate 把 Pod 拦在队列里等待放行，如果放行信号永远不来，兜底机制该怎么设计？超时设长与设短分别会引爆什么？
79. MIG 切分过细为什么会走向调度死锁或资源滞销？请给出一条从碎片到死锁的链路。
80. Gang 调度消灭了"部分调度"的浪费，却制造了碎片的新形态——这种碎片长什么样？为什么装箱打分越高它越疼？
81. VPA 改 Request、HPA 改副本数，两者同时启用为什么可能互相踩踏？描述一次"扩容—缩容—再扩容"循环如何形成。
82. etcd 的 compaction 与 defrag 为什么"定期做"也可能把集群做挂？压缩、碎片整理与空间配额告警的先后顺序该怎么排？
83. 滚动更新期间 PDB 与 HPA 的相互作用：为什么一次普通发版可能触发"驱逐—扩容—再驱逐"的循环？

84. 量级估算：把 10GB 模型权重拆成独立镜像层，对推理冷启动的真实贡献有多大？如何设计实验把"分层"与"预热"的功劳分开？
85. 取舍辩论：日志采集、服务网格与模型初始化各自适合 sidecar 还是进程内 SDK？为什么镜像一大，sidecar 的账就先爆？
86. 假设删掉 Pause 容器，Pod 的网络与进程命名空间会以什么方式坏掉？
87. 节点 NotReady 但 kubelet 进程明明活着——列三个候选原因并按可能性排序。
88. 量级估算：3000 节点、每节点 30 个 Pod、1000 个 Service 的集群，iptables 模式下每条 Endpoint 变更触发的规则重写量级有多大？IPVS 从哪一步开始赚回优势？
89. 取舍辩论：把训练 RDMA 流量与业务流量放进同一张 Overlay 网络，谁先付出代价？RDMA 为什么常被排除在 Overlay 之外？
90. 把所有工作负载的 Limit 都设成 Request 的两倍看起来"更安全"，为什么节点的资源压力驱逐反而更容易发生？
91. 假设 kubelet 只保留 Device Plugin 的资源计数、不再上报 GPU 使用指标，GPU 共享与超卖的哪些问题会变得不可见？
92. 取舍辩论：训练任务用 StatefulSet 还是裸 Pod 加 Operator 管理？两个阵营最硬的论据各是什么？
93. Pod 一直 Pending，但监控显示集群有空闲 GPU——列三个候选原因并按可能性排序。
94. 量级估算：5000 节点、15 万 Pod、平均每 Pod 5KB，etcd 中负载对象的数据量级有多大？加上 Event 之后谁先撞上空间配额？
95. 取舍辩论：Cilium 的 eBPF 数据面对传统 kube-proxy，在一个内核版本老旧的存量集群里，双方各拿什么牌？
96. 假设删掉调度器的打分阶段（只保留过滤），集群短期看起来一切正常——什么样的负载会先让它露馅？
97. HPA 已扩到 maxReplicas，QPS 却没涨、P99 继续恶化——列三个候选原因并按可能性排序。
98. 取舍辩论：GPU Operator 与节点预装驱动，小集群和大集群为什么会做出相反的选择？"运维半径"该怎么计算？
99. 为什么 Cluster Autoscaler 的扩容动作可能让大模型的 Gang 任务反而等得更久？
00. 取舍辩论：GPU 推理发版选滚动更新还是蓝绿？在什么流量曲线下，双倍资源的蓝绿反而更省钱？

综合设计题

74. ★ 从提交一个 vLLM 推理 Pod 到它开始接收流量，列出 Kubernetes 控制面、数据面、CNI、CSI、Device Plugin、kubelet、containerd 各自参与的关键步骤。

分布式系统（修订版）

覆盖一致性模型、Raft 共识、分布式事务、容错与故障检测、数据分片反熵及 AI 训练存储链路的面试题，共 72 题，标注 ★ 的为面试必会 P0 题。

一致性与复制

★1. 解释 CAP 定理的准确含义，以及为什么它常被误解为“三选二”。 2. 什么是 PACELC？它比 CAP 多刻画了哪一维度的取舍？ ★3. 解释线性一致性、顺序一致性、因果一致性、最终一致性的强弱关系与典型系统。 ★4. 什么是线性一致读？为什么它在分布式存储中实现成本很高？ ★5. 解释租约机制，以及它在分布式锁、Leader 选举和缓存一致性中的作用。 6. 什么是逻辑时钟？Lamport 时钟与向量时钟分别解决什么问题？ 7. 解释 Hybrid Logical Clock，以及它如何兼顾物理时钟与因果顺序。 ★8. 什么是 Quorum？W + R > N 为什么能保证强一致读？ 9. 解释链式复制与主从复制的差异，以及它们对写延迟和读扩展性的影响。 10. 什么是状态机复制？为什么分布式存储系统偏爱把状态机复制作为一致性基础？

Raft 与共识

★11. 什么是 Raft？描述 Leader 选举、日志复制、安全性约束的核心流程。 ★12. Raft 中 term 的作用是什么？为什么旧 term 的日志不能随意提交？ 13. 解释 Raft 的成员变更问题，以及单步变更与联合共识的差异。 14. 什么是 Multi-Raft？它在 TiKV、CockroachDB 这类系统中的价值是什么？ 15. 解释 Paxos 与 Raft 的核心差异，以及为什么工程上更常选 Raft。 ★16. 什么是租约读？它如何避免每次读都走 Raft 日志？ 17. 解释 Follower Read 的原理，以及它如何保证不读到过期数据。 18. 什么是日志复制中的流水线优化？它如何提升跨机房复制吞吐？ 19. 解释快照与日志压缩，以及它们对存储空间和恢复时间的影响。 ★20. 解释 Raft 的 ReadIndex 与 Lease Read 两种线性一致读实现的差异，以及它们各自的适用场景。

事务与 MVCC

21. 解释两阶段提交与三阶段提交，以及它们在分布式事务中的阻塞问题。

22. 什么是 Percolator 模型？它如何基于单行事务实现跨行分布式事务？

23. 解释 MVCC 的原理，以及它如何支撑快照读与无锁读。

24. 什么是时间戳分配器？中心化 TSO 与去中心化 HLC 的取舍是什么？

25. 解释分布式事务中的写偏斜与幻读，以及 SSI 如何检测冲突。

26. 什么是幂等性？在分布式存储和推理网关中如何设计幂等接口？

27. 解释 Exactly-Once 语义的实现难点，以及它为什么通常依赖幂等和事务。

容错与故障检测

★28. 什么是分布式系统中的脑裂？如何通过 Quorum、Fencing、STONITH 等手段缓解？ ★29. 解释 Fencing Token 的作用，以及它在分布式锁和存储访问中的意义。 30. 什么是 Gossip 协议？它在成员管理与故障检测中的优缺点是什么？ 31. 解释 SWIM 故障检测协议，以及它相比心跳机制的优势。

数据分片与反熵

★32. 什么是一致性哈希？它如何解决节点扩缩容时的数据迁移问题？ 33. 解释虚拟节点在一致性哈希中的作用，以及它如何缓解数据倾斜。 34. 什么是 CRDT？它适合哪些最终一致场景？在 AI 集群元数据管理中有何潜在用途？ 35. 解释 Merkle Tree 在分布式存储反熵与数据校验中的应用。 ★36. 什么是纠删码？它相比多副本

在存储成本与恢复开销上的取舍是什么？ 37. 解释副本放置策略中的机架感知、可用区感知与故障域隔离。 38. 什么是数据本地性？在 Kubernetes 调度训练任务时如何利用数据本地性？ 39. 解释数据分片与 Shuffle，以及它们对分布式训练吞吐和存储 I/O 的影响。 40. 什么是数据版本管理与血缘？它们对可复现训练和增量数据管道为什么重要？

分布式文件系统与缓存

★41. 解释缓存一致性协议在分布式缓存中的体现，以及缓存穿透、击穿、雪崩的区别。 42. 什么是 CDN 边缘缓存？它和训练数据分发、模型分发有什么可借鉴的相似性？ ★43. 解释对象存储、块存储、文件存储的差异，以及它们分别适合 AI 训练中的哪类数据。 44. 什么是 S3 一致性模型的历史演变？强一致对象存储对数据管道有什么影响？ 45. 解释 HDFS 的架构与瓶颈，以及为什么现代 AI 训练数据流常转向对象存储 + 缓存层。 ★46. 什么是 JuiceFS、Alluxio、CubeFS 这类分布式文件系统？它们的核心抽象和缓存策略是什么？ 47. 解释 FUSE 在分布式文件系统中的性能瓶颈，以及如何通过内核客户端或 FUSE 旁路优化。 48. 什么是数据预取？在训练数据加载中，预取如何与 GPU 计算重叠？ 49. 什么是列式存储？Parquet、ORC 的设计如何影响训练数据读取效率？ 50. 解释小文件问题，以及它在对象存储和分布式文件系统中对训练数据加载的影响。 51. 解释分布式文件系统中元数据服务的扩展瓶颈，以及分片与多 Raft 化的扩展策略。

思考与取舍题

53. Raft 已经有租约读这种"便宜"的线性一致读，为什么不干脆用它满足一切读，把 ReadIndex 留作备用？
54. 构造一个满足 W+R>N 却读不到最新写入的具体场景。
55. etcd 内嵌了 Raft，性能瓶颈却常落在 fsync 与 watch 上——这个现象说明什么？它对"选对共识算法就解决了一致性"的预期提出了什么修正？
56. Multi-Raft 的热点 region 迁移"已经完成"，为什么 P99 的抖动窗口远长于迁移本身？
57. 量级估算：纠删码修复单个坏块的读放大是三副本方案的多少倍？为什么"省一半存储"的账在小文件场景经常不成立？
58. 从 3 副本迁移到纠删码，容量账确实省了——哪些隐性成本最容易被漏算？
59. 一致性哈希解决了扩缩容的数据迁移，但网关拿它做会话保持后新增了什么问题？"粘住同一节点"与"无状态网关"为什么会打架？
60. 脑裂场景下 Fencing Token 能拦住旧主，但 token 校验本身也可能"晚到"——构造一个晚到窗口造成破坏的场景。
61. 如果业务要求 30 秒 RTO，follower 的追赶速度该如何倒推 Raft 的日志保留时长与快照频率？
62. 取舍辩论：Raft 写入需要多数派，三节点与五节点的可用性、写延迟、成本账各怎么算？什么场景下"两副本加见证节点"比三副本更优？
63. 假设删掉 Raft 的日志匹配检查，系统会以什么方式悄悄损坏数据而不是立即报错？
64. Leader 稳定、无网络分区，某个 follower 的 apply 延迟却持续高于同伴——列三个候选原因并按可能性排序。
65. 量级估算：1000 QPS、每条 1KB 的写入，一个 5 副本 Raft 组的磁盘写流量与网络流量分别是业务流量的几倍？
66. 强一致存储的读吞吐可以靠加 follower 扩展，写吞吐为什么几乎不能？

67. MVCC 旧版本回收做激进还是保守？激进会伤到什么读者、保守会拖垮什么？哪种负载组合会让版本垃圾爆炸？
68. 假设把 Raft 日志刷盘改成"每秒批量 fsync 一次"，正常时吞吐暴涨——故障发生时哪些安全性被破坏？什么业务能接受这个代价？
69. 对象存储已经强一致，下游数据管道却偶尔读到"旧文件"——列三个候选原因并按可能性排序。
70. 取舍辩论：分布式锁选 Redis 还是 etcd？从脑裂窗口、租约续期与延迟三个维度辩论，什么业务才值得为强一致付费？
71. 量级估算：跨机房部署（单向 RTT 30ms）的 Raft 组，一次提交至少要多久？commit 等待与 fsync 为什么是叠加而不是并行？
72. Gossip 发现节点故障要几秒，Raft 选举只要百毫秒——两层故障检测的时间差会制造什么混乱？到底该谁说了算？

综合设计题

★52. 从一次分布式训练任务读取样本开始，到样本进入 GPU 显存，列出可能涉及的元数据服务、缓存层、存储后端、网络传输与本地 I/O 关键环节。

存储内核面试题库（修订版）

覆盖 I/O 栈、文件系统、存储引擎、SSD/NVMe、FUSE 与容器存储的面试题，共 73 题，标注 ★ 的为面试必会 P0 题。

系统调用与页缓存

★1. 从一次 read 系统调用出发，描述数据从磁盘到用户态缓冲区的完整路径。 ★2. 从一次 write 系统调用出发，描述数据从用户态到持久化介质的完整路径。 ★3. 解释 page cache 的读缓存与写缓存分别如何工作。 4. 什么是 writeback？脏页什么时候被回写，如何调优。 5. 解释 dirty_ratio 与 dirty_background_ratio，它们对训练 checkpoint 写入有什么影响。 6. 什么是回写抖动？为什么大量写入可能拖慢读延迟。

块层与 I/O 调度

7. 解释 buffer head 与 bio 的关系，以及块层如何合并 I/O 请求。
8. 什么是 I/O 调度器？none、mq-deadline、bfq、kyber 分别适合什么场景。
9. 解释 blk-mq 多队列块层，以及它如何适配 NVMe 高并发。
10. 什么是 I/O 合并与 I/O 拆分？它们如何影响吞吐与延迟。

文件系统与日志

★11. 解释 direct I/O 的语义，以及它和 page cache 的交互。 ★12. 什么是 O_SYNC、O_DSYNC、fsync、fdatasync 的差异。 13. 解释文件系统的日志模式：journal、ordered、writeback。 14. 什么是日志提交延

迟？它如何影响小文件写入性能。 15. 解释 ext4 的 extent 树与延迟分配。 16. 解释 XFS 的 allocation group 与并行分配优势。 17. 解释 Btrfs 的 CoW 与快照机制，以及它对写放大的影响。

存储引擎

★18. B+ 树与 LSM-tree 相比，各自的读放大与写放大来自哪里？如何根据读写比例与空间预算做取舍？ 19. 解释 LSM-tree 的 leveled compaction 与 tiered compaction，它们在读放大、写放大与空间放大上分别如何权衡？ ★20. 解释 bitcask 模型：全内存哈希索引、追加写日志文件与定期合并如何协作？它适合什么样的负载？ 21. 解释 WiscKey 的键值分离思想：它如何把键与值分开存储以利用 SSD 的顺序写特性，又引入了哪些新的代价？ 22. 对比 crash consistency 的三条实现路径：journal（日志）、shadow paging（影子页）与 COW（写时复制），各自的原理、运行时开销与崩溃恢复方式是什么？

SSD 与 NVMe

★23. 什么是写放大？它在 SSD、RAID、CoW 文件系统中分别如何产生。 ★24. 解释 SSD 的 FTL、垃圾回收、TRIM、磨损均衡。 25. 什么是 SSD 的写放大系数？如何估算与降低。 26. 解释 NVMe 队列深度与并发，如何判断设备是否成为瓶颈。 27. 什么是 NVMe polling 模式？它相比中断模式的优势与 CPU 代价。 28. 解释 SPDK 的原理，以及它适合哪些存储加速场景。

高性能 I/O 与内核旁路

★29. 什么是 io_uring？它的 submission queue 与 completion queue 如何工作。 30. 解释 io_uring 的 SQPOLL、IORING_SETUP_IOPOLL、registered buffers。 31. io_uring 相比 epoll + AIO 在训练数据加载中的优势是什么。 32. 什么是 O_DIRECT 与 AIO 的组合？它适合训练数据读取吗？ 33. 什么是内核旁路？它在存储与网络中的典型实现有哪些。

FUSE 与用户态文件系统

★34. 解释 FUSE 的请求路径，以及它为什么可能成为分布式文件系统的瓶颈。 35. 什么是 FUSE 的 writeback cache 与 direct_io 模式。 36. 解释 virtio-fs 与 FUSE 的差异，以及它在容器场景中的价值。

元数据与路径查找

37. 什么是文件系统的元数据操作？为什么小文件训练数据集对元数据压力大。

38. 解释 inode 缓存与 dentry 缓存，它们如何影响小文件读取。

39. 什么是负 dentry？它如何加速不存在路径的查找。

40. 解释 open 路径查找的完整流程，以及如何用 open_by_handle_at 绕过。

预读、mmap 与大页

★ 41. 什么是文件预读？readahead 如何影响顺序读训练数据。 42. 解释 fadvise 的 POSIX_FADV_WILLNEED、DONTNEED、SEQUENTIAL、RANDOM。 ★43. 什么是 mmap 的缺页路径，以及 major fault 与 minor fault 的差异。 44. 解释 mmap 写时复制与 msync 的语义。 ★45. 什么是大页？透明大页 THP 对数据库和 AI 数据加载的利弊。 46. 解释 hugetlbfs 与显式大页的使用场景。

缓存层次与镜像设备

47. 解释块设备缓存与文件系统缓存的重叠问题。

48. 什么是 loop 设备？它在容器镜像与文件系统镜像中的角色。

容器与 Kubernetes 存储

★49. 解释 overlayfs 的读路径与写路径，以及它对容器启动和模型加载的影响。 50. 什么是 overlayfs 的 copy-up？它如何影响首次写入性能。 51. 解释容器中 volume 挂载的传播模式：shared、slave、private、rslave。 52. 什么是 CSI 的 node plugin？它如何把远程存储挂载到 Pod。 53. 解释本地 PV 与 hostPath 的差异，以及本地缓存的隔离风险。 54. 解释分布式文件系统客户端的读缓存一致性模型。

思考与取舍题

56. 同一块 NVMe 上，XFS 在什么条件下处理海量小文件反而会输给 ext4？给出至少两个可量化的条件并说明判断依据。

57. THP 对延迟敏感型服务是双刃剑：请各举一个"打开后更糟"与"关闭后更糟"的真实场景，并说明该看哪些指标做裁决。

58. io_uring 的 registered buffers 需要锁定物理内存：估算一次 8 GB 数据加载要 pin 住的内存量级，并讨论它与 cgroup 内存限制及 OOM killer 的冲突风险。

59. NVMe 队列深度打满后，吞吐仍在上升而 P99 已经陡增：这个拐点的位置由什么决定？如果你只能选一个工作点，会选在哪里？

60. FUSE 的 writeback cache 模式下，应用读到"已写入"的数据与远端真实状态之间存在一致性窗口：这个窗口受什么控制？对多客户端共读一个数据集的训练任务，你敢开吗？

61. 模型权重在容器里通常被假设只读：什么操作会触发 overlayfs copy-up 打破这个假设？若平台每天启动数百个容器加载同一个 20 GB 权重文件，copy-up 的代价发生在哪里？

62. 缓存层的写放大与 SSD 磨损均衡叠加后，一块标称 3 DWPD 的盘在高频覆写场景可能提前多久到达寿命极限？辩论一下：缓存放内存、放本地盘、放远端，磨损代价分别转移到了哪里？

63. 用 direct I/O 绕过 page cache 后，内核预读也随之失效：这块工作该由应用自己做异步预取，还是干脆放弃顺序性假设？为训练数据读取的两种路线辩论一次。

64. 现象诊断：训练任务第一个 epoch 很快，之后每个 epoch 稳定变慢。候选原因：(a) 数据集远大于内存，page cache 命中率随扫描轮次下降；(b) checkpoint 写入累积引发回写抖动；(c) 元数据缓存被逐出使 open 路径变慢。请排序并说明各需要什么证据。

65. 假设破坏：如果未来介质让顺序写与随机写一样快，LSM-tree 相对 B+ 树还剩下哪些优势？哪些取舍会整个重排？
66. 调大 dirty_ratio 能让写入更"平滑"，为什么反而可能让 checkpoint 完成时间的尾部更差？在训练与在线服务混部的机器上，你会投票调大还是调小？
67. 现象诊断：fio 测得顺序读 6 GB/s，但训练数据加载只有 1 GB/s。候选原因：(a) 应用读取块太小且未对齐；(b) 预读窗口与真实访问模式不匹配；(c) 每文件 open/close 的元数据与路径查找开销。请排序并设计一个最小实验区分它们。
68. 取舍辩论：一个数据集通常只被完整读一遍——这种负载下 page cache 还剩多少价值？"direct I/O + 用户态自管缓存"是否是更诚实的选择？
69. 量级估算：一亿个 1 KB 小文件落在 4 KB 块大小的 ext4/XFS 上，仅数据块与 inode 的占用下限是多少？这个数字如何改变"小文件该不该打包"的争论。
70. 现象诊断：同一容器内，dd 写大文件速度正常，应用写同一目录却慢一个数量级。候选原因：(a) 应用是小块同步写且逐笔 fsync；(b) 该路径发生了 overlayfs copy-up；(c) 容器 cgroup 的 I/O 限速。请排序并给出各项的验证手段。
71. 反直觉追问：把预读窗口从 128 KB 加大到 8 MB，顺序读吞吐可能提升，什么时候反而变差？
72. 量级估算：磁盘顺序读 7 GB/s、内存 256 GB、数据集 500 GB 且训练按随机顺序访问：估算长稳训练时的有效数据供给带宽；缓存命中率从 90% 提到 99% 又能换来多少？
73. 量级估算：估算一次 fsync 在消费级 NVMe 与企业级 NVMe 上的时间量级，并推算单线程每秒最多完成多少笔"写 + fsync"；这个数字如何改变日志类系统的设计取舍。

综合设计题

★55. 如果让你为训练数据读取设计一个单机存储加速层，你会如何组合 page cache、direct I/O、io_uring、预读、本地 NVMe 缓存与 FUSE 旁路？

网络内核面试题库（修订版）

覆盖内核网络收发路径、TCP 连接与拥塞控制、无损网络与 RDMA 的面试题，共 41 题，标注 ★ 的为面试必会 P0 题。

数据包路径与内核收发

★1. 从一次 socket send 出发，描述数据从用户态到网卡的完整路径。 ★2. 从一次网卡收包出发，描述数据从硬件中断到用户态 read 返回的完整路径。 3. 解释 NAPI 与轮询机制，它如何缓解高包速率下的中断开销。 4. 什么是软中断与 ksoftirqd？它们在高吞吐网络场景中如何成为瓶颈。 ★5. 解释 GRO、GSO、TSO、LRO 的作用与差异。 6. 什么是 RSS 与 RPS？它们如何把网络负载分散到多核。 7. 解释 XPS 与 IRQ affinity，如何避免网卡中断集中在一个 CPU。 8. 解释 XDP 的处理位置与 eBPF 网络加速路径，它为什么能在驱动层完成丢包、重定向与转发？

零拷贝与内核旁路

★9. 什么是零拷贝网络路径？sendfile、splice、MSG_ZEROCOPY 分别如何工作。 10. 解释 DPDK 的原理，以及它适合哪些网络加速场景。

TCP 连接管理

★11. 解释 TCP 三次握手在内核中的状态机与队列：SYN queue、accept queue。 12. 什么是 SYN flood？syncookies 如何缓解。 ★13. 解释 TIME_WAIT 的成因，以及大量 TIME_WAIT 对推理网关的影响。 14. 解释 tcp_tw_reuse 与 tcp_tw_recycle 的历史与风险。 15. 解释 SO_REUSEPORT 的内核分发机制，它如何在多个进程间实现连接级负载分担，并支持服务的无缝重启与滚动升级？

TCP 流量控制与拥塞控制

★16. 解释 TCP 滑动窗口、接收窗口、发送窗口与零窗口探测。 17. 什么是 Nagle 算法与延迟确认的交互，为什么它们会叠加出延迟。 ★18. 解释 TCP 拥塞控制状态机：慢启动、拥塞避免、快速重传、快速恢复。 ★19. 解释 CUBIC 与 BBR 的核心思想差异，以及它们对跨机房训练数据流的影响。 ★20. 什么是 ECN？它如何与 DCTCP、DCQCN 配合实现低延迟无损网络。

RDMA 与无损网络

★21. 解释 PFC 的原理，以及它可能引发的 Head-of-Line Blocking 与死锁。 ★22. 什么是 RDMA？它相比 TCP 在内核旁路、零拷贝、CPU 卸载上的优势是什么。 23. 解释 RDMA verbs 的核心对象：PD、MR、QP、CQ。 24. 什么是 RoCEv2？它为什么需要无损网络，运维难点在哪里。 25. 解释 InfiniBand 的 subnet manager 与 fabric 基本概念。 ★26. 什么是 GPUDirect RDMA？它如何让网卡直接读写 GPU 显存。

思考与取舍题

28. 零拷贝在什么情况下是负优化？给出小包高频场景中 sendfile/MSG_ZEROCOPY 反而更慢的条件，并辩论"省下的拷贝成本"实际被转移到了哪里。
29. BBR 在广域网上常常占优，为什么在数据中心内不一定赢过 CUBIC？当瓶颈是交换机队列而非带宽时，两者的行为差异会如何放大或缓解 incast？
30. 现象诊断：跨 NUMA 部署后网络吞吐掉一半，CPU 利用率却不高。候选原因：(a) RSS 队列与处理核不在同一 NUMA 节点；(b) 收包缓冲落在远端内存拖慢拷贝；(c) 中断集中在单个核上。请排序并给出验证手段。
31. GRO 用增加一跳延迟换吞吐：在 10 Gbps 与 100 Gbps 网卡上，这个交换的临界点分别在哪里？对小包 RPC 服务你开还是不开？
32. PFC 死锁的触发条件是什么？辩论一下：一个从未出过死锁的无损网络，究竟是"配置正确"还是"条件还没凑齐"？
33. syncookies 开启后，TCP 握手的哪些信息会丢失或被降级？对一个依赖这些选项的推理网关，攻击结束后应不应该立刻关掉它？

34. 长短连接混合的推理网关出现数万 TIME_WAIT：把下面威胁按真实程度排序并说明理由——(a) 临时端口耗尽导致无法建连；(b) 内核连接哈希查找变慢；(c) 内存占用拖垮主机。
35. 现象诊断：网关每天固定时段 P99 抖动，抓包可见集中重传。候选原因：(a) 定时批量任务引发 incast；(b) 监控采集抢占软中断 CPU；(c) 上游交换机缓存溢出丢包。请排序并说明你先查哪个计数器。
36. 反直觉追问：把网卡 ring buffer 调到最大后丢包反而变多——给出至少两种机制上的解释。
37. 取舍辩论：内核旁路（DPDK/XDP）占用的 CPU 核若留给推理服务还能多跑请求：在什么利用率水平下旁路才划算？
38. 量级估算：100 Gbps 链路上 1500 字节包的每秒包数是多少？若每包处理预算要压进 1 微秒，需要几颗核专职收包？据此评估"内核协议栈够不够用"这句话。
39. 假设破坏："带宽充足时，拥塞控制算法对延迟没有影响"——用一次 incast 场景击碎这句话，并指出此时真正稀缺的资源是什么。
40. 现象诊断：RDMA 集合通信偶发秒级超时，交换机上有 PFC pause 帧但零丢包。候选原因：(a) PFC 水线配置不当引发下游风暴；(b) Head-of-Line Blocking 在某条链路累积；(c) 对端 GPU 侧停顿反向传导。请排序并说明排序依据。
41. SO_REUSEPORT 的连接分发看似均匀，为什么实际负载常集中于少数 worker？在不改内核的前提下，你在调度层面有什么补救思路？

综合设计题

27. 如果让你为分布式训练设计一个跨节点数据与梯度传输的网络层，你会如何组合 TCP、RDMA、拥塞控制、拓扑感知与故障降级？

数据流加速（修订版）

面向大模型训练数据管道方向的面试题库，覆盖数据路径与瓶颈定位、GPU 拷贝重叠、解码预处理、数据格式与列存、缓存与数据局部性调度、Checkpoint 加载、网络传输与尾延迟可观测，共 105 题；★ 前缀为该方向面试必会题。

> **分层使用说明（2026-10-06 起生效）**
>
> - **★ = P0**：训练端到端数据路径、page cache 行为、本地缓存与调度、对象存储 range 读、可观测性与延迟建模——与 P1 项目直接咬合；
> - **P1 概念级**：数据格式（WebDataset/MDS/Arrow）、分布式文件系统缓存策略、预取/双缓冲/列存谓词下推——知道做什么用、对调度有什么影响即可；
> - **P2 跳过**：DALI/nvJPEG/NVDEC/音频/视频解码加速器（17–26）——GPU 侧解码不是平台岗主场，P1 §9.4 已明确概念边界，本节只需知道"DALI 把 CPU 解码移走、不影响跨网络取数"；

数据路径与瓶颈定位

★1. 从一次训练 step 出发，描述数据从存储到 GPU 显存的完整路径，并标出所有可能阻塞的环节。

★2. 什么是 GPU 数据加载瓶颈？如何判断瓶颈在 CPU、内存、PCIe、网络、存储还是框架层？

★3. 解释 GPUDirect Storage 的原理，以及它绕过了哪些传统数据通路环节。

4. GPUDirect Storage 对文件系统、驱动、GPU 架构分别有什么要求？

5. 什么是弹性数据加载？当存储带宽波动时如何动态调整预取深度？

6. 什么是数据加载的端到端 profiling？如何用 Nsight、PyTorch Profiler、eBPF 联合定位瓶颈？

DataLoader 与 GPU 拷贝重叠

★7. 解释 DataLoader 的工作原理，以及 num_workers、prefetch_factor、pin_memory 分别影响什么。

8. 为什么 DataLoader 的 worker 不是越多越好？worker 数量增加后可能引入哪些新瓶颈？

9. 什么是 H2D 拷贝？CUDA Stream（默认流与多流）与 CUDA Event 如何在数据加载与计算重叠中协作，并做精确计时？

★10. 什么是异步预取？如何在 Go 或 Python 中实现可控的异步数据预取？

★11. 解释双缓冲与多缓冲机制，以及它们在数据加载 pipeline 中的应用。

★12. 什么是 backpressure？数据加载 pipeline 中如何避免生产者压垮消费者？

13. 解释环形缓冲区在数据加载与流式处理中的价值。

14. 什么是零拷贝序列化？它在跨进程数据加载中的典型实现方式有哪些？

15. 解释共享内存在大规模数据加载中的优势与同步挑战。

16. 大 batch 训练对数据加载吞吐与显存的系统压力是什么？系统层如何配合调整？

解码与预处理加速「P2」

> GPU 侧解码加速属算法/框架工程师主场。平台岗只需：①知道 DALI 是"GPU 上做解码+增强"、②能答清它**不解决跨网络取数问题**（P1 的 t1 仍然独立成立）、③知道常用数据集格式为 WebDataset/MDS 即可。**第一轮直接跳过。**

17. 什么是 NVIDIA DALI？它试图解决数据加载中的哪些 CPU 瓶颈？

18. 解释 DALI 的 pipeline 执行模型，以及它与 PyTorch DataLoader 的差异。

19. 什么是数据增强？它应该在 CPU、GPU 还是专用加速器上执行？

20. 解释 GPU 数据增强的收益与代价，以及它和 CPU 数据增强的 pipeline 差异。

21. 什么是 nvJPEG、nvJPEG2000？它们如何加速图像解码？

22. 解释视频解码在训练数据流中的瓶颈，以及 NVDEC 的作用。

23. 什么是音频解码与重采样瓶颈？如何用 GPU 或专用库加速？

24. 解释文本 tokenization 的 CPU 开销，以及如何做批量 tokenization 加速。

25. 什么是 tokenizer 并行化？它在大规模语言模型训练数据流中为何重要？

26. 解释数据去重与过滤在训练数据流中的位置，以及它们对存储 I/O 的影响。

27. 什么是流式数据集？它相比离线数据集在数据加载上有何不同？

28. 解释在线数据增强与离线数据增强的取舍。

数据格式与列存

★29. 解释 WebDataset 格式的设计动机，以及它为什么适合大规模训练数据流。

★30. 什么是 sharded dataset？分片粒度如何影响顺序读、随机读和负载均衡？

31. 解释 TFRecord、RecordIO、WebDataset、Parquet 在训练数据读取中的取舍。

32. 什么是列式存储的谓词下推？它如何减少训练数据读取量？

33. 解释 Parquet 的 row group、column chunk、page 结构，以及它们对预取的影响。

★34. 什么是 Arrow？它为什么适合零拷贝数据交换和跨语言数据管道？

35. 什么是数据压缩？压缩率、解压 CPU 开销与读取带宽之间如何取舍？

36. 解释 zstd、lz4、gzip 在训练数据存储中的适用场景。

37. 什么是列式压缩？Parquet 的编码与压缩如何影响读取吞吐？

38. 解释字典编码与 run-length encoding 在训练数据读取中的价值。

39. 什么是数据格式的 zero-copy 读取？哪些格式更适合零拷贝？

40. 解释 Arrow IPC 与 Flight，以及它们在分布式数据流中的角色。

本地与分布式缓存

★41. 什么是数据预热？如何在训练开始前把热数据预取到 page cache 或本地 NVMe？

★42. 解释本地 NVMe 缓存在训练数据流中的作用，以及如何做缓存淘汰。

★43. 什么是缓存命中率？如何针对训练数据访问模式设计缓存策略？

44. 解释 LRU、LFU、ARC、2Q 等缓存淘汰算法在训练数据场景中的适用性。

45. 什么是存储客户端缓存？FUSE 客户端缓存与内核客户端缓存有何差异？

46. 解释 JuiceFS 的元数据与数据分离架构，以及它如何影响数据流加速。

47. JuiceFS 元数据引擎（如 Redis、TiKV、SQL 数据库）的选型如何影响数据流性能与可靠性？

48. 什么是 Alluxio 的层次化存储？它如何做数据本地性与缓存编排？

49. 解释 CubeFS 的元数据、数据节点与副本策略。

★50. 数据集版本化如何与缓存失效保持一致？数据变更后如何避免训练读到新旧混杂的缓存？

51. 预热缓存与按需缓存的取舍是什么？启动延迟与首轮命中率如何权衡？

52. 训练场景“单写多读”的缓存一致性假设何时失效？失效后需要什么机制补救？

数据局部性与调度

53. 什么是数据加载中的随机性？如何保证分布式训练中随机种子与分片的一致性？

54. 解释 sampler 与 sharding 的关系，以及 epoch 边界如何影响数据读取模式。

★55. 什么是数据局部性？训练数据分片如何与节点、机架、GPU 拓扑对齐？

★56. 解释数据亲和性调度，以及它如何减少跨网络数据读取。

★57. 什么是 cache-aware 调度？Kubernetes scheduler plugin 如何感知节点上的缓存位置并据此打分？

58. 什么是存储 QoS？在多租户 AI 集群中如何避免某个训练任务打满存储带宽？

59. 解释 cgroup blkio 与 io.max 对训练数据读取的限速作用。

60. 本地 NVMe 缓存如何做空间配额与多任务争抢控制？cgroup io.max 在这里有哪些局限？

★61. 训练数据管道的混沌测试如何设计？应注入哪些故障、验证哪些指标？

Checkpoint 与模型加载

★62. 解释 checkpoint 写入对训练数据流的冲击，以及如何做异步 checkpoint。

★63. 什么是 checkpoint 分片？它如何提升大模型检查点写入和恢复速度？

64. 解释模型加载路径：从对象存储到 GPU 显存，中间经过哪些缓存与拷贝。

★65. 什么是模型权重预取？如何在推理服务启动时隐藏模型加载延迟？

66. 解释 mmap 加载模型权重的优缺点，以及它和 safetensors 的关系。

67. 什么是 safetensors？它相比 pickle 在安全性和加载性能上有什么优势？

68. 解释量化模型加载中的数据流变化，以及反量化发生在哪个环节。

69. checkpoint 恢复的懒加载如何设计？为什么权重优先加载、优化器状态后台拉取能缩短恢复时间？

★70. 什么是对象存储的 multipart upload？它如何加速大文件写入和检查点保存？

网络传输

71. 解释 NCCL 在数据流与集合通信中的角色，以及它和存储 I/O 的边界。

72. 什么是梯度压缩？它如何减少通信量，又可能引入哪些数据流问题？

73. 什么是网络带宽延迟积？它如何影响跨机房训练数据读取效率？

74. 解释 TCP 窗口与拥塞控制对高吞吐数据读取的影响。

75. 解释多路径网络与 ECMP 在 AI 集群数据流中的价值。

★76. 解释 S3 的 range 读，以及它如何支持分片并行读取训练数据。

77. 什么是 Arrow Flight SQL？它相比 JDBC/ODBC 在高吞吐数据传输中的优势是什么？

78. 解释 gRPC 流式传输在大规模数据加载中的优缺点。

79. 什么是 HTTP/2 多路复用？它如何影响数据下载并发？

80. 解释 QUIC 在跨广域网数据同步中的潜在优势。

可观测与尾延迟

★81. 什么是数据管道可观测性？应该采集哪些延迟、吞吐、队列深度指标？

82. 解释 tracing 在数据加载 pipeline 中的作用，以及如何定位慢环节。

★83. 什么是 P99 延迟？为什么数据加载的尾延迟会拖慢整体训练吞吐？

★84. 解释 straggler 问题，以及数据分片不均衡如何造成训练慢节点。

思考与取舍题

86. 预取深度调到 32 后，固定 shuffle 种子的基准吞吐提升 40%，换一种数据顺序就跌回原样甚至更差。为什么会出现"对 page cache 的过拟合"？如何设计一个不被数据顺序愚弄的评测方法？

87. 预取太浅喂不饱 GPU，太深又会挤占带宽和内存。同一台机器上"单任务独占"与"多任务共享"两种部署，为什么最优预取深度差很多？在线判断"喂不饱"还是"冲过头"该看哪个信号？

88. shuffle 越彻底模型质量越好，磁盘顺序 I/O 却越差。给出至少两种"保住全局 shuffle 又保住顺序读"的工程折中，并说明各自牺牲了什么。

89. 估算：图像数据集 5 亿张、平均 300KB（约 150TB），训练要把本地 NVMe 读带宽打满 6GB/s。全局均匀 shuffle 下，缓存池要占到数据集多大比例才能稳住 90% 命中率？"首轮训练"这个约束又如何改变结论？

90. cache-aware 调度把任务尽量钉在已有缓存的节点上，为什么在数据集远大于缓存池时反而制造热点过载与冷节点闲置？出现什么信号时，应该主动把任务派往零缓存的节点？

91. 弹性扩容在负载尖峰时才加节点，预热却要在训练开始前就把数据搬进本地盘，两者在时间轴上天然冲突。给出至少两种让扩容节点快速"变热"的折中，并指出各自的带宽代价。

92. 数据集从 v3 灰度升级到 v4，新旧任务并行期间同一份数据可能在缓存里存两套。以"版本号进缓存 key"为前提，双份缓存的成本该谁承担？什么情况下应该放弃灰度、强制全量失效？

93. 异步 checkpoint 把写压力挪到后台，但触发后的几分钟里数据读取 P99 明显恶化。列出 3 个候选原因并按可能性排序，并说明哪个原因决定了你应该把 checkpoint 写到另一块物理盘。

94. 把海量小文件合并成大分片换来了顺序吞吐，但"随机访问单条样本"与"流式断点续读"的代价会发生什么变化？描述一个明显不适合合并的访问模式。

95. 列存谓词下推省下的读取量，什么时候会被"按文件或 row group 粒度整块缓存"的浪费吃掉？缓存粒度应该跟着格式结构走，还是跟着访问模式走？

96. 给训练任务配了 io.max 限速后，平均带宽确实被保护了，但单步数据加载 P99 从 80ms 涨到 1s。解释限速器在什么工作模式下会把"平均限速"变成"脉冲式排队"，限速值该按平均速率还是峰值速率设？

97. GPUDirect Storage 与 DIO 绕过 page cache 后，内核预取随之失效，"预取"该由谁接手？用户态预读、存储侧缓存、GPU 侧三种承接方式的适用边界分别是什么？

98. 假设把 page cache 完全关掉、所有读走直通路径：哪种数据访问模式几乎无损，哪种会崩溃式变差？请用"同一样本两次被读到的间隔"刻画这条分界线。

99. 缓存命中率稳定在 95%，训练 step 的数据加载 P99 却仍打到秒级。列出 3 个能让"高命中率"失去意义的机制，按最容易被忽略的程度排序。

00. 存储与网络带宽每代翻倍，解压 CPU 却涨得慢。为什么"该不该压缩"的结论会随硬件代际漂移？举一个"五年前正确、现在可能错误"的压缩决策。

01. 估算：训练每 step 消费 256 个样本、step 时间 400ms，单样本解码加增强平均 10ms、P99 约 50ms，DataLoader 开 8 个 worker。平均供给能力够吗？把 P99 样本的影响算进去后，结论还成立吗？

02. 为消灭 straggler 把分片切得绝对均匀后，全局 shuffle 的质量为什么会受损？负载均衡与随机性冲突时，训练的不同阶段该分别优先保哪边？

03. 多机训练里只有个别节点周期性读慢，且"慢节点"随时间漂移。列出 3 个候选原因并按可能性排序，指出你第一个要采集的信号。

04. 对象存储 range 读在"64 并发 × 8MB"时打满带宽，改成"16 并发 × 32MB"后掉了一半——总在途字节相同，结果为什么不同？请从拥塞窗口、单流上限与存储端分片布局三个角度说理。

05. GPUDirect Storage 免掉了经 CPU 的中转拷贝，但要求驱动栈、内存对齐与内核版本全链路配合。什么时候"多一次拷贝"的旧路径反而是更优的工程决策？给出至少三个成立条件。

综合设计题

85. 如果让你设计一个面向大模型训练的“数据流加速层”，你会从存储、缓存、预取、格式、网络、GPU 通路哪些维度给出架构原则？

推理引擎优化（修订版）

面向大模型推理方向的面试题库，覆盖 vLLM 架构与源码、KV Cache 管理、批处理调度、并行通信、投机解码、PD 分离与高并发网关，共 133 题；★ 前缀为该方向面试必会题。

> **分层使用说明（2026-10-06 起生效）**
>
> - **★ = P0**：TTFT/TPOT、continuous batching、KV Cache 管理（PagedAttention、prefix caching、offload）、网关路由与限流、弹性扩缩容、PD 分离架构权衡——与 P2 项目直接咬合；
> - **P1 概念级**：投机解码、Tensor Parallel、pipeline parallel、LoRA 热加载、容量规划与成本模型——知道做什么、对调度/成本有什么影响即可；
> - **P2 跳过**：量化推理（33–34）、FlashAttention 原理、kernel fusion、CUDA Graph、动态 shape、vLLM 多实例 GPU 部署细节（35–45）——推理引擎 kernel/驱动层工程师主场，平台岗只需知道"FlashAttention 降低访存、CUDA Graph 降低 launch 开销、量化降显存但可能影响精度"；

推理基础与指标

★1. 解释大模型推理的两个阶段：prefill 与 decode，它们的计算特征和瓶颈有何不同？

★2. 为什么 prefill 阶段通常 compute-bound，而 decode 阶段通常 memory-bandwidth-bound？

★3. 解释 TTFT 与 TPOT 的定义，它们分别受哪些系统层因素影响？从网关、调度、KV Cache、批处理角度如何优化首 token 延迟？

4. 解释吞吐与延迟的权衡，在推理服务中如何做分级 SLA？

批处理与调度

★5. 什么是 continuous batching？它相比静态 batching 的优势是什么？

6. 解释 vLLM 的调度器如何决定每一步哪些请求进入 batch。

7. 解释 prefill 与 decode 混合批处理，以及它为什么能提升 GPU 利用率。

8. 什么是 chunked prefill？vLLM 调度器中它如何与 decode 优先级协同？它如何改善长 prompt 下的延迟与吞吐？

9. 什么是请求批处理窗口？动态批处理与连续批处理有何差异？冷热模型分层如何影响批处理？如何根据负载自动调优批大小以平衡延迟与吞吐？

10. 解释抢占与重计算，在显存不足时 vLLM 如何处理请求？

11. 解释 beam search 在推理服务中的系统开销，以及如何做批处理优化。

12. 什么是 sampling 参数对 decode 性能的影响？

13. 解释 logits processor 在推理 pipeline 中的位置与开销。

14. 什么是推理服务的请求隔离？如何避免长请求阻塞短请求？

15. 解释推理服务中的公平调度与权重公平队列。

16. 什么是推理服务的 GPU 时间片与优先级抢占？

17. 什么是推理请求的 deadline 传播？如何避免超时请求继续占用 GPU？

KV Cache 管理

★18. 什么是 PagedAttention？它如何借鉴虚拟内存思想管理 KV Cache？

★19. 解释 KV Cache 的 block 管理与 paged 布局，block size 的选择如何影响显存碎片、显存带宽利用与吞吐？

★20. 解释 Prefix Caching 的命中条件与 hash 机制（block hash 如何识别可复用前缀），LRU 淘汰如何与之协作？它在多轮对话和 RAG 场景中的价值是什么？

★21. 什么是 KV Cache 量化？它如何降低显存占用，又可能损失什么？

★22. 解释 KV Cache offload 到 CPU 内存或 NVMe 的取舍、换入换出机制与 vLLM swap 的触发条件，如何避免换入换出成为 PCIe 瓶颈？

23. 解释 vLLM 的 block manager 与 scheduler 如何协作。

24. 解释显存碎片？如何通过分配器或 PagedAttention 缓解？

25. 解释 CUDA 内存池在推理服务中的作用。

26. 什么是 GPU 显存超分？在推理服务中是否可行？

27. 解释推理服务的跨节点 KV Cache 复用可行性。

★28. prefix cache 命中率应如何测量？网关侧估计与引擎自报指标有何差异，各有什么坑？

并行与通信

29. 什么是 tensor parallel？它如何切分模型并影响通信？vLLM 部署在多 GPU 与跨节点时采用哪些通信模式？

30. 解释 pipeline parallel 在推理中的气泡问题，以及 vLLM 对它的支持现状与调度难点。

31. 什么是 expert parallel？MoE 推理中它带来哪些调度挑战？

32. 解释 MoE 推理的 all-to-all 通信瓶颈。

33. 什么是量化推理？INT8、INT4、FP8 分别适合什么场景？

34. 解释 GPTQ、AWQ、SmoothQuant 在系统层的差异。

35. 解释 attention kernel 的 FlashAttention 优化原理。

36. 什么是 FlashDecoding？它如何优化长上下文 decode？

37. 解释 CUDA Graph 在推理中的应用，以及它如何降低 kernel launch 开销。

38. 什么是 kernel fusion？它如何减少显存读写？

39. 解释推理中的动态 shape 问题，以及它如何影响 CUDA Graph 捕获。

40. 解释多 GPU 推理的通信瓶颈。

41. 什么是 NVLink 域内通信与跨域通信的差异？

42. 解释推理服务中的负载均衡与 GPU 亲和性。

43. 什么是 GPU 拓扑感知调度？NVLink、PCIe、NUMA 如何影响推理性能？

44. 解释 NVIDIA MPS 的原理与适用场景。

45. 解释推理服务的多实例 GPU 部署与副本间负载均衡。

投机解码

★46. 什么是 speculative decoding？它如何用 draft 模型加速 decode？Medusa、EAGLE 等方法在系统层有何差异？draft 与 target 模型如何协同调度（含 vLLM 的集成方式）？

47. 投机解码对 TBT（token 间隔时间）指标有什么影响？为什么平均 TPOT 改善时 TBT 抖动反而可能变差？

网关・流式・发布

48. 解释推理网关的核心职责：路由、鉴权、限流、重试、超时、可观测性。

49. 什么是加权路由？在多个推理后端之间如何做负载均衡？

50. 解释一致性哈希在推理会话保持中的作用。

51. 什么是会话亲和性？多轮对话中为什么可能需要它？

★52. KV 感知路由：推理网关为什么需要知道 KV Cache 的位置？缓存位置注册与租约机制应如何设计？

★53. 解释推理网关中的排队策略：FIFO、优先级队列、公平调度。

54. 什么是请求优先级与抢占？如何避免低优先级请求饿死？

★55. 解释推理网关的限流算法：令牌桶、漏桶、滑动窗口。

56. 什么是并发限制与队列深度控制？它们如何保护后端 GPU？

57. 解释推理网关的重试策略，以及为什么盲目重试会放大故障。

58. 什么是熔断与降级？在推理服务中如何设计？

59. 解释 gRPC 与 HTTP 在推理网关中的取舍。

★60. 什么是 SSE 与 WebSocket？流式推理输出分别适合哪种协议？

61. 解释 HTTP/2 多路复用对流式推理的影响。

62. 什么是背压传播？网关如何把后端压力传递到客户端？

63. 解释请求超时在流式推理中的特殊性。

64. 什么是 token 级流式输出？它如何影响网关缓冲策略？

65. SSE 流式转发有哪些工程坑？flush 时机、客户端取消传播、禁用压缩、超时重置分别怎么处理？

66. 解释推理网关中的请求排队与 vLLM 内部队列的协同。

67. 解释推理服务的 token 预算与最大并发 token 数控制。

68. 多租户推理服务如何设计 token 预算与限流？请覆盖账户级、请求级、token 级三层控制与超卖策略。

69. 解释金丝雀发布与蓝绿发布在推理服务中的差异。

70. 什么是模型版本灰度？如何在网关层做流量切分？

71. 解释多模型服务中的模型加载与卸载策略。

72. 什么是模型缓存？如何在节点本地缓存常用模型权重？

73. 什么是模型预热？如何在扩容后快速进入可服务状态？

74. 推理服务弹性扩缩容的冷启动问题如何解决？从镜像预拉取、模型预热、CUDA Graph 预捕获三条路径分别说明。

75. 解释推理服务的节点本地模型缓存与共享内存加载。

76. 解释模型加载时的显存峰值问题。

可观测与成本

77. 解释推理服务的可观测性指标：TTFT、TPOT、QPS、并发数、KV Cache 使用率。

78. 什么是分布式追踪？如何在网关、后端、GPU 之间串联 trace？

79. 解释 Prometheus 指标在推理服务中的典型采集方式。

★80. 什么是 GPU 利用率？为什么它可能高但吞吐仍然低？

81. 解释 SM 占用率与显存带宽利用率的关系。

82. 解释推理服务的尾延迟来源：排队、批处理、KV Cache 换入换出、网络。

83. 什么是推理服务的 SLO？如何把它拆解到系统层指标？

84. 解释推理成本优化：GPU 利用率、批大小、量化、缓存、弹性。

85. 解释推理服务的成本模型？如何计算每百万 token 的 GPU 成本？

86. 如何搭建每百万 token 的成本模型？需要计入哪些变量（缓存命中折扣、批处理效率、闲时摊销等）？

87. 解释推理服务的容量规划：如何根据 QPS、TTFT、TPOT 估算 GPU 数量？

88. 推理服务的容量规划与压测如何设计？闭环压测与开环压测的区别是什么，各自适合验证什么指标？

★89. 为什么 KV cache 剩余量比 GPU 利用率更适合作为推理服务弹性伸缩的信号？基于它做 HPA/KEDA 扩缩容要解决哪些问题？

vLLM 架构与源码

★90. 解释 vLLM V1 引擎架构相比早期版本的关键变化。

91. 什么是 vLLM 的 EngineCore 与 AsyncLLMEngine 的职责分离？

92. 解释 vLLM 的多 LoRA 服务，以及 LoRA 适配器切换对批处理的影响。

93. 什么是 LoRA 热加载？如何在网关层路由到不同适配器？

94. LoRA 多适配器服务的路由与热切换如何设计？适配器注册、路由表更新与切换期间的请求如何做到无损？

95. 什么是 Triton Inference Server？它的 backend、ensemble、dynamic batching 如何工作？

96. 解释 Triton 的模型仓库与版本管理。

97. 什么是 TensorRT-LLM？它相比 vLLM 的取舍是什么？

98. vLLM Production Stack、AIBrix、llm-d、NVIDIA Dynamo 这类推理服务框架各自定位是什么？在自研与开源之间如何取舍？

PD 分离

★99. 解释 prefill-decode 分离（disaggregated prefill/decode）架构，它如何让两类资源独立扩缩容？

★100. 解释 KV Cache 在 prefill 节点与 decode 节点之间的传输问题。

101. 什么是 KV Cache 传输的压缩与量化？

思考与取舍题

04. 把动态批处理的等待窗口从 5ms 调大到 50ms，吞吐上升、平均 TTFT 几乎不变，P99 TTFT 却明显恶化——为什么对均值无害的调整会伤到尾部？这个窗口的平衡点该用到达强度还是用分位数指标来定？
05. 连续批处理号称"新请求随到随进批"，为什么一个 32k 上下文的长请求仍能把同批短请求的 TPOT 拖垮？"按请求数公平"与"按 token 数公平"两种口径，各会让谁吃亏？
06. 投机解码的接受率掉到多少以下会变成负优化？为什么批越大这个临界值越高——能不能用 draft 与 target 的算力比写出一条判断规则？
07. 把 chunked prefill 的块大小从 512 调到 8192，TTFT 与 TBT 各会滑向哪个极端？"块越大越接近完整 prefill"在什么负载下成立、什么负载下失效？
08. 开启 chunked prefill 后 TTFT 改善，但 TBT P99 翻倍而 TPOT 均值不变。列出 3 个候选原因并按可能性排序，指出每个原因独有的验证信号。
09. MIG 或 vGPU 把显存切成各占一块的硬边界后，PagedAttention 在实例内消灭的碎片为什么会在实例间重新出现？什么负载画像下，切分后的整卡可服务并发反而低于不切分？
10. 假设把 PagedAttention 退回"按 max_model_len 预分配连续 KV"：对上下文两极分化（要么几百、要么 32k）的流量，显存有效利用率会掉到什么量级？请用并发数与长度分布说理。
11. prefix cache 命中要求把同前缀请求粘在同一副本，这与负载均衡天然冲突：什么场景该牺牲命中率保均衡，什么场景反过来？各举一例并说明判断信号。
12. prefix cache 命中率从 40% 提到 85%，每百万 token 的账单却几乎没降。列出至少 3 个让"命中"不等于"省钱"的机制，并指出哪个最常被成本模型漏掉。
13. 把 gpu-memory-utilization 从 0.90 调到 0.95，OOM 反而更频繁。请给出一条从"水位线调高"到"进程被杀"的完整原因链，并说明它与调高 GOMEMLIMIT 后反而更容易分配失败为什么是同一类反噬。
14. 估算：70B 稠密模型（80 层、GQA 8 个 KV 头、head_dim 128），权重 FP8、KV Cache FP8，单个 32k 上下文请求的 KV 占多少显存？一张 80G 卡扣除权重与激活后能并行服务几路这样的请求？
15. 估算：KV Cache offload 到 CPU 内存后，按 PCIe Gen4 x16 约 32GB/s 的有效带宽，把 40GB KV 换回显存需要多久？换入若发生在请求恢复路径上，什么样的 TTFT SLO 会让"offload 省显存"直接不成立？
16. 估算：排队等待近似正比于 ρ/(1-ρ)，利用率从 50% 拉到 90% 时等待时间涨了大约多少倍？据此说明容量规划里"平均延迟 × 目标 QPS"式线性外推为什么会系统性低估突发需求。
17. PD 分离让 prefill 与 decode 各自独立伸缩，为什么在中小规模流量（比如日处理几十亿 token 以下）常常是负优化？KV 跨节点传输、链路故障面、两个池的错峰闲置，哪个最先吃掉分离收益？
18. SLO 拆成 TTFT 与 TPOT 两条独立分位数线后，调度器每一步都要在"让新请求插队做 prefill"与"保 decode 批不被打断"之间二选一。为什么这两条 P99 在固定算力下近似互斥？该看什么信号决定当前偏向哪边？
19. 估算：后端 P99 处理 30s、客户端超时 10s、失败重试 2 次、峰值 1000 QPS。一次 60s 的后端卡顿最坏会给后端制造多大的在途请求洪水？给出算式，并说明有效到达率被放大了几倍。
20. PD 分离上线后 TTFT 全面改善，TPOT 却从 15ms 涨到 40ms，decode 节点 GPU 利用率反而下降。列出 3 个候选原因并按可能性排序。

21. 全链路无报错、限流未触发、GPU 利用率不高，TTFT P99 却从 800ms 涨到 8s。给出 3 个候选原因，按"最可能且验证成本最低"排序。
22. 每百万 token 的账单成本与真实 GPU 成本之间常有 30% 以上的缺口。列出至少 4 个隐性成本项，并指出哪一项会随并发非线性放大。
23. 抢占重算是显存不足的兜底，但重算烧的是 prefill 算力：一个 8k 上下文的请求被反复抢占 3 次，相当于白跑多少 prefill？什么负载画像下"宁可排队或拒绝也不抢占"总成本更低？
24. 假设把推理网关从链路里拿掉、客户端直连推理引擎：哪些能力立刻缺失，哪些本来就该做在客户端？流量规模小到什么程度时，网关的剩余价值撑不起它引入的延迟与故障面？
25. 多租户 token 预算的超卖依赖"租户峰值不同时出现"的统计复用。什么事件会让这个假设瞬间失效？失效时按付费等级还是按已消耗预算决定牺牲顺序，各自的风险是什么？
26. prefix cache 采用 LRU 淘汰时，一次 100k token 的单发长文档请求为什么会冲掉大量高频系统提示前缀？淘汰权重应该按 block 数量定，还是按未来复用期望定？
27. 估算：RAG 流量固定前缀 4k token、命中率 90%、峰值 200 QPS，一天省下多少 prefill token？按 7B 模型单卡数万 token/s 的 prefill 吞吐折算成 GPU 秒，再与"少租一张卡"的账对比。
28. 假设彻底去掉抢占、显存不足就拒绝新请求：哪些体验反而变好，哪些场景会系统性变差？"准入拒绝"与"抢占重算"各自适合什么结构的 SLO？
29. 基于 KV cache 剩余量的自动扩缩容上线后，副本数每天震荡 4～6 次，每次震荡伴随 TTFT 尖峰。列出 3 个候选原因并按可能性排序。
30. INT4 权重量化让单卡塞得下两倍副本，但吞吐未必翻倍、延迟未必变好。"多副本"与"单副本大 batch"是两种花显存的方式，各自在什么请求形态下占优？
31. TP 从 4 加到 8 缓解了显存压力、缩短了单步时延，为什么 decode 阶段吞吐反而可能下降？每层通信次数随卡数增长而每步计算量变小，这笔账在什么上下文长度下最不划算？
32. 估算：decode 每生成一个 token 都要把全部 KV 读一遍，128k 上下文的单 token 生成比 8k 上下文多付多少倍的带宽代价？这如何让"按 token 统一定价"与真实成本倒挂？
33. 令牌桶放行突发、漏桶强行整形：对连续批处理引擎，突发流量该被抹平还是被放行？从凑批收益与排队延迟两个角度，说明什么条件下该选那个"看起来错"的。

综合设计题

★102. 如果让你设计一个面向 vLLM 的 Kubernetes 推理平台，你会如何组合网关、调度、KV Cache 管理、弹性扩缩容与多租户隔离？

★103. 如果让你从零设计一个“vLLM + Kubernetes + 高并发网关”的推理基础设施，你会如何划分控制面、数据面、缓存层、调度层与可观测性层？

ML 系统基础面试题库（修订版）

面向 AI Infra **平台岗**（Go + 云原生 + 训练/推理基础设施）的 ML 系统基础题库，覆盖训练与推理、分布式并行、显存与通信、采样与批处理的系统问题，共 60 题。

> **分层使用说明（2026-10-06 起生效，按岗位定位重新标层）**
>
> - **★ = P0 必会**：能脱稿讲 60–120 秒，从系统指标与工程取舍角度答，不背数学推导；
> - **P1 概念级**：节后标注「P1」的小节——能讲清"是什么、影响哪项系统指标（显存/带宽/通信/延迟）"即可，不深挖实现；
> - **P2 算法侧**：节后标注「P2」的小节——属算法工程师主场，平台岗面试**只需知道存在与边界**，本库中保留仅为知识索引，第一轮直接跳过；
> - 答题统一口径：**平台岗答 ML 题的满分姿势是"系统账"**（显存字节数、通信量、带宽利用率、对调度的影响），而不是复述算法论文。

训练与推理基础

1. 解释训练与推理在计算图上的本质差异。
2. 什么是前向传播与反向传播？反向传播在系统层需要保存哪些中间状态？
3. 什么是计算图？静态图与动态图对系统优化分别意味着什么。
4. 解释算子融合的数学依据，以及它为什么能同时省显存和带宽。
5. 什么是梯度？为什么分布式训练要频繁做梯度同步。

显存与混合精度

6. 解释优化器状态：SGD、Adam、AdamW 分别需要保存什么，显存开销差多少。
7. ★ 什么是混合精度训练？FP16、BF16、FP8 的数值范围与系统取舍是什么。
8. 解释 loss scaling 的作用，以及它为什么在 FP16 训练中必要。
9. 什么是激活值？为什么激活值显存常成为大模型训练的瓶颈。
10. 什么是 gradient checkpointing？它用计算换显存的代价是什么。

分布式并行与集合通信

11. ★ 解释数据并行、张量并行、流水线并行的切分维度与通信模式。
12. ★ 什么是 ZeRO 的三个阶段？它分别切分优化器状态、梯度和参数。
13. ★ 解释 AllReduce、AllGather、ReduceScatter 的数学语义与通信量。
14. 什么是集合通信的 ring 与 tree 算法？它们分别适合什么规模。
15. 解释通信与计算重叠的常见实现方式。
16. 什么是 MoE？它的路由机制对系统负载均衡提出什么要求。
17. 解释 expert parallel 中 all-to-all 通信为什么可能成为瓶颈。

量化与数值精度「P1」

> 平台岗口径：知道量化把显存/带宽占用降多少、对 kernel 与精度各有什么代价即可；GPTQ/AWQ 等具体算法差异不考（08 章有服务侧视角的对应题）。

18. 什么是量化？PTQ 与 QAT 的系统差异是什么。
19. 解释权重量化与激活量化的区别，以及它们对 kernel 的影响。

KV Cache、采样与批处理

20. ★ 什么是 KV Cache？它缓存的是什么，为什么 decode 阶段必须依赖它。
21. 解释注意力计算的复杂度，以及长上下文对显存和带宽的压力。
22. 解释采样：greedy、temperature、top-k、top-p 对推理系统负载的影响。
23. 什么是 beam search？它为什么比 greedy 更吃显存和算力？early stopping 又如何影响系统负载？
24. 如果算法同学说“把 batch size 调大一点”，你会从显存、通信、数据加载、收敛性哪些系统层面评估这个改动？

模型结构与系统开销「P2」

> 平台岗面试几乎不直接考模型结构的数学细节。本节的正确用法是：遇到算法面试官描述模型结构时，你能听懂、能将其翻译成显存/带宽/通信账，**不主动讲**

25. 解释 embedding 层与输出层的参数量关系，以及权重共享的系统意义。
26. 什么是词表大小对显存和采样开销的影响。
27. 解释 LayerNorm、RMSNorm 在系统层的计算与显存访问差异。
28. 什么是残差连接？它对反向传播和显存有什么影响。
29. 解释 Transformer 的层数与隐藏维度如何影响显存、算力和通信。
30. 什么是 attention head？多头注意力在系统层如何并行。
31. 解释位置编码：绝对位置、相对位置、RoPE 对推理缓存的影响。
32. 什么是 ALiBi？它相比 RoPE 在长上下文推理中的系统差异。

注意力变体与 KV Cache 优化

33. ★ 解释 KV Cache 的数学含义，为什么它只缓存 K 和 V。
34. ★ 什么是 GQA 与 MQA？它们如何减少 KV Cache 显存。
35. 解释 MHA、MQA、GQA 在推理吞吐上的差异。
36. 什么是滑动窗口注意力？它如何降低长上下文显存。
37. 解释稀疏注意力与系统实现难点的关系。

MoE 系统设计

38. 什么是 MoE 的 top-k 路由？它如何影响批处理与负载均衡。
39. 解释 MoE 的容量因子与 token 丢弃问题。

微调、蒸馏与稀疏化「P2」

> 训练侧微调算法不属平台岗主场。**例外：LoRA 在多 LoRA 推理服务场景是高频题**（适配器热加载/路由/批处理），该视角 08 章第 92–94 题已覆盖，本节只需知道 LoRA 是"低秩适配器、省显存、可插拔"即可。

40. 什么是 LoRA？它在系统层如何减少训练显存与适配器存储。
41. 解释 QLoRA 的量化与反量化发生在哪些环节。
42. 什么是蒸馏？它在推理系统层带来的收益与代价是什么。
43. 解释剪枝与稀疏化对 kernel 和硬件利用率的影响。

解码加速与训练稳定性

44. ★ 解释 draft model 与 target model 的接受率如何影响加速比。
45. ★ 解释大 batch 与学习率调度变化对显存、数据加载与系统稳定性的影响。

思考与取舍题

> 分层：P0 工程账必做 **48、50、51、54、55、57、60**（显存/通信/批处理/调度账）；P1 概念推理 47、49、52、53、58、59；P2 算法侧跳过 **56**（loss 曲线诊断属训练调参）。

47. 混合精度常被总结成"更快更小"，但从 loss scaling、梯度溢出检查到优化器精度，系统上多出了哪些联动环节？哪个环节会先成为瓶颈？
48. 量级估算：参数量翻倍，训练显存需求为什么远不止翻倍？请把参数、梯度、优化器状态与激活值的账拆开算一遍。
49. GQA 把 KV Cache 显著压缩之后，什么场景下 attention 的计算反而变慢？
50. 一个推理服务要挂上百个 LoRA 适配器，批处理的隐性上限最先卡在哪类资源上？
51. ZeRO-3 用通信换显存：把互联带宽从 400Gb 换成 800G，这个交换的盈亏拐点大约移动多少？拐点前后分别该改用什么并行方式？
52. beam 宽度与批容量是"抢座位"的关系：beam 开大后谁被挤出批？为什么吞吐反而可能下降？
53. 用重算换显存时，被重算的部分往往享受不到算子融合与缓存命中——为什么实际显存省得比理论少、时间花得比理论多？
54. 量级估算：7B 模型 BF16 推理，单个 token 的 KV Cache 有多少字节？上下文从 4K 拉到 128K，显存大头如何搬家？

55. 假设删掉 KV Cache、每一步重算全部历史的 attention，decode 吞吐会掉到什么量级？这种"最坏方案"反而适合什么特殊场景？
56. 训练 loss 曲线出现周期性锯齿、GPU 利用率同步波动——列三个候选原因并按可能性排序。
57. 为什么 8 卡训练把 batch size 从 32 提到 256，每 token 成本可能不降反升？
58. 模型量化到 INT8 后吞吐没有上涨——给出一条从 kernel 到访存的诊断顺序，判断是算子没换还是瓶颈根本不在带宽。
59. 假设把 expert parallel 的 all-to-all 从 RDMA 改为经 CPU 中转的 TCP，MoE 训练会先在哪里露馅？
60. continuous batching 让吞吐翻倍，首 token 延迟却可能恶化——两者争抢的是什么资源？什么流量构成下应该主动限制批规模？

综合设计题

46. 如果让你从系统层向算法团队解释“为什么这个模型训练慢/推理贵”，你会如何拆解参数量、激活值、KV Cache、通信、数据加载与硬件利用率？

性能工程面试题库（修订版）

覆盖性能分析方法论、基准测试、Profiling 工具与系统排查工具链的面试题，共 45 题，标注 ★ 的为面试必会 P0 题。

性能分析方法论

★1. 什么是性能工程中的“测量优先”原则？为什么凭直觉优化往往适得其反？ ★2. 解释 USE 方法与 RED 方法，并说明它们分别适合排查哪类系统。 3. 如何系统化定位一个训练任务的瓶颈在 CPU、内存、I/O、网络还是 GPU？ 4. 什么是 Amdahl 定律？它如何限制系统整体加速比。 5. 什么是 Universal Scalability Law？它比 Amdahl 定律多刻画了什么。 ★6. 解释排队论中的 Little 定律，以及它如何用于推理网关容量估算。 ★7. 什么是尾延迟？为什么 P99 比平均值更能反映用户体验。 ★8. 解释尾延迟放大效应，一个请求经过多级服务时尾延迟如何累积。 9. 什么是抖动？如何区分 GC 抖动、调度抖动、网络抖动与存储抖动。

基准测试与负载建模

★10. 解释微基准测试的陷阱：预热、编译器优化、缓存效应、测量开销。 11. 什么是负载生成？如何设计贴近真实推理请求分布的压测。 ★12. 解释闭环压测与开环压测的差异，以及它们分别适合什么场景。 ★13. 什么是协调遗漏？为什么基于响应时间的压测可能低估系统压力。

Profiling 工具

14. 解释 perf 的基本用法，如何用 perf top、perf record、perf report 定位热点。 ★15. 什么是火焰图？on-CPU 火焰图与 off-CPU 火焰图分别揭示什么。 ★16. 解释 eBPF 在性能分析中的优势，以及 bpftrace 的典型使用场景。

15. 什么是 ftrace 与 tracepoint？它们在内核延迟分析中的价值是什么。
16. 解释 strace、ltrace 的开销，以及何时不该用它们。

GPU 与 AI 负载分析

19. 什么是 Nsight Systems？它如何展示 CPU、GPU、CUDA API 的时间线。
20. 什么是 Nsight Compute？它如何分析单个 kernel 的瓶颈类型。
21. 解释 PyTorch Profiler 的用法，以及它如何与 Nsight 互补。

生产性能工程

22. 什么是持续 profiling？它如何在生产环境中低开销采集性能数据。
23. 解释性能回归检测，如何把 benchmark 集成到 CI 中。
24. 什么是容量规划？如何根据 QPS、P99、GPU 利用率估算所需副本数。

系统排查工具链

★25. 解释 top、mpstat、vmstat 的关键指标：%sys、%soft、run queue 分别说明什么？读数异常时分别指向哪类问题？ ★26. 如何用 free 的 available 字段与 major fault 定位内存压力与回收、换页问题？ ★27. 解释 iostat 的 util、await、aqu-sz，以及它们与 IOPS、带宽的关系，如何判断磁盘是否打满？ ★28. 如何用 ss 与 ethtool -S 观察重传、丢包与队列溢出计数？ ★29. 如何用 /proc/interrupts 与 IRQ affinity 排查中断集中在单个 CPU 的问题？ ★30. 如何用 pidstat 定位 DataLoader worker 的 CPU 与 I/O 占用异常？ ★31. 黑盒决策树：一个训练任务突然变慢，你只有一台机器上的 shell，给出完整的排查顺序与每一步的判断依据。 ★32. 什么是基准测试卫生？为什么需要多次运行取中位数、报告方差并锁定环境变量？

思考与取舍题

34. 把 P99 打下来可能损害 P50：构造一个"优化了长尾、伤害了中位数"的具体优化例子，并辩论这种交换在什么业务上值得做。
35. 闭环压测里调出来的"最优批大小"，上线后为什么常常失效？从开环/闭环差异出发，列出至少两个只有上线才会暴露的因素。
36. 火焰图采样率过低时会制造"虚假热点"：请描述这种假象的成因，并设计一个实验判断某张火焰图里的头号热点是真是假。
37. 压测达标、上线雪崩：假设协调遗漏已经排除，还有哪些环节在"骗"你的压测报告？至少说出三种。
38. 同一个 benchmark 首轮与第十轮结果相差 20%：列出至少三种解释，并设计最小实验把它们区分开。
39. 容量规划用均值而非分布会系统性低估资源：以一个均值 80 ms、P99 500 ms 的服务为例，估算按均值规划的集群在高峰时会差多少？这个误差该由哪个指标来背锅？
40. 现象诊断：重构后 perf 热点函数换了，端到端耗时却没变。候选原因：(a) 瓶颈本来就在 off-CPU；(b) 新热点只是旧瓶颈的下游影子；(c) 内存带宽已成为硬上限。请排序并说明判断证据。

41. 反直觉追问：把最热函数优化掉 30%，整体吞吐只涨 2%——除了 Amdahl 定律，还有哪些现实因素在稀释收益？
42. 取舍辩论：压测环境要不要完整复刻生产的缓存预热状态？接受冷启动数据与精确预热，哪种结论对上线更有指导性？
43. 量级估算：10k QPS、P99 目标 100 ms 的服务，每分钟有多少请求"天然"超过 100 ms？摊到一万个用户身上，单个用户一天会遇到几次？这个数字如何影响你对 SLO 投诉的处理优先级。
44. 假设破坏：如果 profiler 自身有 5% 开销、且恰好改变了锁的时序，你还能相信它给出的"锁争用"结论吗？怎样设计验证才能自洽？
45. 现象诊断：GPU 利用率显示 95%，吞吐却只有理论值的三分之一。候选原因：(a) 利用率统计把"有 kernel 在跑"当作"在算有用功"；(b) 大量小 kernel 的 launch 间隙未被计入；(c) 拷贝与计算串行化挤占了计算时间。请排序并说明你还想再要哪张图。

综合设计题

★33. 如果让你为一个 vLLM 推理服务做端到端性能优化，你会如何组织测量、定位、假设、验证、回归这套闭环？

Python 面试题库（修订版）

覆盖 GIL 与并发模型、内存管理、C 扩展与 PyTorch 数据加载的面试题，共 45 题，标注 ★ 的为面试必会 P0 题。

GIL 与并发模型

★1. 解释 CPython 的 GIL 是什么，它保护了什么，又不保护什么。 2. 为什么 GIL 对 CPU 密集型多线程程序影响巨大，但对 I/O 密集型程序影响较小？ ★3. 解释 Python 多线程、多进程、asyncio 三种并发模型的适用场景与代价。 4. 什么是 asyncio 事件循环？它如何调度协程，以及和 Go netpoll 的异同。 5. 解释 async/await 的本质，协程在什么时刻会让出控制权。 6. 什么是阻塞事件循环？哪些常见调用会阻塞 asyncio，如何避免？ 7. 解释 asyncio 中的 Task、Future、Coroutine 的区别。 8. 什么是 asyncio 背压？如何用 Semaphore、Queue 控制并发。 9. 解释 Python 多进程中的 fork、spawn、forkserver 启动方式差异。

内存管理与序列化

★10. 在 PyTorch DataLoader 中，num_workers > 0 时数据是如何跨进程传递的？ ★11. 解释 Python 对象序列化 pickle 的开销，以及它如何影响 DataLoader 吞吐。 12. 什么是共享内存？Python 中如何用 shared_memory 或 mmap 降低数据拷贝？ 13. 解释 Python 内存管理：引用计数、循环 GC、内存池 pymalloc。 14. 什么是内存泄漏在 Python 中的表现？如何用 tracemalloc、objgraph 排查？ 15. 解释 Python 对象的 **slots** 的作用，以及它对内存和访问速度的影响。 16. 什么是弱引用？它在缓存和避免循环引用中的价值是什么。

C 扩展与 CUDA 交互

17. 解释 Python 的 GIL 在 C 扩展中的释放，为什么 numpy、PyTorch 能绕过 GIL 并行。
18. 什么是 C 扩展？pybind11、ctypes、cffi 各自的适用场景与开销。
19. 解释 Python 调用 CUDA 的路径，以及同步、异步调用对训练数据加载的影响。
20. 什么是 CUDA Stream 与 Python 异步的交互，如何避免不必要的同步。

PyTorch DataLoader

21. 解释 PyTorch DataLoader 的 collate_fn 的作用，以及它可能成为瓶颈的原因。 ★22. 什么是 pin_memory？在 DataLoader 中它如何与 CUDA 拷贝配合？ ★23. 解释 DataLoader 的 prefetch_factor 与 num_workers 的协作关系。
22. 什么是 IterableDataset？它相比 Map-style Dataset 在流式数据场景中的优势是什么。 ★25. 解释 PyTorch Sampler 与 DistributedSampler 的工作原理，以及 epoch 边界如何处理。
23. 什么是 DataLoader 的 worker 重启？它如何影响训练稳定性。

服务性能与工程实践

27. 解释 Python logging 在高并发推理服务中的性能陷阱，如何做异步日志。
28. 什么是 Python 的 GIL 争用？如何用多进程或 C 扩展规避。
29. 解释 Python 的垃圾回收对低延迟推理服务的影响，如何调优 gc 参数。
30. 什么是 Python 的 frozenset、tuple、dict 在哈希与内存上的差异。
31. 解释 Python 生成器与迭代器的惰性求值，它在流式数据管道中的价值。
32. 什么是 context manager？它在资源释放与 CUDA 上下文管理中的作用。
33. 解释 Python 的异常处理开销，以及热点路径中如何避免异常驱动控制流。
34. 什么是 Python 的 typing 与运行时开销，typing 是否影响推理性能。

思考与取舍题

36. DataLoader 的 num_workers 超过物理核数后反而变慢：描述这条性能曲线的机制，并说明如何区分它是"过度订阅 CPU"还是"内存带宽已打满"。
37. 一个 100 MB 的 tensor 走 pickle 跨进程传输：它实际会经历哪几次内存拷贝？哪一步最容易被忽略？
38. C 扩展释放 GIL 后线程真正并行了，CPU 反而可能成为新瓶颈：从内存带宽与缓存竞争的角度，辩论"线程数 = 核数"这条经验还剩多少效力。
39. fork 出的子进程里，共享内存对象与普通对象的引用计数分别会发生什么？这个差异如何在"看起来正常、跑一天就泄漏"的现象里埋雷？
40. gc.freeze 在 DataLoader worker 启动时到底冻结了什么？在什么负载特征下它值得调用，什么情况下只是安慰剂？

41. 现象诊断：多进程训练的常驻内存随 epoch 单调上涨。候选原因：(a) fork 后 COW 页被写触碰逐渐私有化；(b) shared_memory 段创建后未正确 close/unlink；(c) worker 内的缓存容器无界增长。请排序并说明你会先抓哪个证据。
42. 反直觉追问：把预处理逻辑从 Python 改写成 C 扩展后，wall time 降了、CPU 总占用却升了——这一定是坏事吗？
43. 取舍辩论：batch 跨进程传递用 pickle 走管道，还是用 shared_memory 加环形缓冲？请在 batch 很小与 batch 很大两种极端下分别辩论一次。
44. 量级估算：batch 为 256×3×224×224 的 float32、每秒 50 个 batch 走 pickle 进管道：估算管道需要承载的字节率，并判断瓶颈会先出现在序列化 CPU 还是管道带宽。
45. 假设破坏：如果 CPython 明天彻底移除 GIL，现有"多进程 DataLoader + 主进程聚合"的架构里哪些设计要推倒重审？哪些理由依然成立？

综合设计题

★35. 如果让你优化一个 PyTorch 训练数据加载 pipeline，你会从 GIL、多进程、共享内存、序列化、pin memory、CUDA Stream 哪些角度入手？

C++ & CUDA 面试题库（修订版）

覆盖 C/C++ 内存模型与并发、编译链接、CUDA 执行模型与性能分析的面试题，共 53 题，标注 ★ 的为面试必会 P0 题。

> **分层使用说明（2026-10-06 起生效，按岗位定位重新标层）**
>
> - **P0 必会（★）**：C/C++ 内存布局、对齐、malloc/RAII/智能指针、atomic 内存序、SIMD、编译链接（1–26）——平台岗写 Go 之外，K8s 组件、存储引擎、CUDA 组件的 C/C++ 依赖都需要这层常识；
> - **P1 概念级**：CUDA 执行模型、Stream、统一内存（27–36）——P1 的 §9.4 知识边界题已覆盖概念，这里只要求"知道 Stream 解决什么、为什么统一内存会有页迁移抖动"；
> - **P2 跳过**：CUDA kernel 优化、CUDA Graph、occupancy、bank conflict、maxrregcount、CUDA Graph 捕获限制（37–40、42–53）——CUDA kernel 工程师主场，平台岗只需知道"FlashAttention/CUDA Graph 存在，降低 launch 开销"这一句话级结论；
> - **岗位分工一句话**：平台岗考 CUDA 的上限是"知道数据怎么进 GPU、Stream 怎么重叠"，下限是"别去和 CUDA 工程师拼 kernel 优化"。

C/C++ 内存模型与数据布局

★1. 解释 C 的内存布局：栈、堆、BSS、data、rodata 各自存放什么。 2. 什么是未定义行为？举例说明它为什么可能让编译器做出反直觉优化。 3. 解释指针与数组的区别，以及数组退化为指针时丢失了什么信息。 4. 什么是指针对齐？为什么未对齐访问在某些架构上会崩溃或变慢。 ★5. 解释结构体内存对齐与 padding，以及如何通过重排字段减小结构体大小。 ★6. 什么是缓存友好的数据布局？AoS 与 SoA 分别适合什么访问模

式。 7. 解释 C 的 volatile 关键字，它和原子操作、内存屏障的区别是什么。 8. 什么是 restrict 指针？它如何帮助编译器优化。

内存管理与 C++ 抽象

★9. 解释 malloc/free 的底层实现思路，以及常见分配器如 jemalloc、tcmalloc 的差异。 10. 什么是内存池？在高吞吐数据管道中为什么常用内存池替代频繁 malloc。 ★11. 解释 C++ RAII，以及它如何保证异常安全与资源释放。 12. 什么是移动语义？std::move 真的移动了什么？ 13. 解释左值、右值、完美转发，以及它们对零拷贝封装的意义。 ★14. 什么是智能指针 unique_ptr、shared_ptr、weak_ptr？各自开销与适用场景。 15. 解释 shared_ptr 的引用计数线程安全性，以及它不能保证什么。 16. 什么是虚函数表？虚函数调用相比普通函数调用多出哪些开销。 17. 解释 C++ 模板的编译期展开，以及它如何实现零开销抽象。 18. 什么是 CRTP？它在高性能库中如何避免虚函数开销。

并发与原子操作

★19. 解释 std::atomic 的内存序：relaxed、acquire、release、seq_cst。 20. 什么是无锁队列？ABA 问题如何产生，如何用 tagged pointer 或 hazard pointer 解决。 21. 解释 false sharing 在 C/C++ 中的表现，以及如何用 alignas 缓解。

编译、链接与向量化

★22. 什么是 SIMD？编译器自动向量化需要满足哪些条件。 23. 解释编译链接过程：预处理、编译、汇编、链接各自做什么。 24. 什么是静态链接与动态链接？它们对容器镜像大小和启动速度的影响是什么。 25. 解释符号可见性与 -fvisibility=hidden 在大型 C++ 项目中的作用。 26. 什么是 ABI？为什么 C++ ABI 兼容性比 C 更难保证。

CUDA 执行模型与显存访问

★27. 解释 CUDA 的线程层次：thread、warp、block、grid 的关系。 28. 什么是 warp divergence？它如何影响 kernel 性能。 ★29. 解释 CUDA 的 shared memory、bank conflict，以及如何优化访问模式。 30. 什么是 CUDA occupancy？寄存器压力、共享内存如何限制并发 warp 数。 ★31. 解释 CUDA 的 global memory 合并访问，什么样的访问模式最差。

CUDA 内存管理

★32. 什么是 CUDA Stream 与 Event？如何用它们实现拷贝与计算重叠。 33. 解释 cudaMalloc 与 cudaMallocAsync 的差异，以及显存池的价值。 34. 什么是统一内存？它在训练与推理中的优缺点是什么。 35. 解释 GPU 页错误与 UVM 迁移，为什么它可能造成性能抖动。 36. 解释 cudaHostRegister 注册内存的原理与代价，以及 GPUDirect Storage 对内存注册（预先注册的固定缓冲区）提出了哪些要求？

Kernel 优化与性能分析

★37. 什么是 CUDA Graph？它如何降低 kernel launch 开销，捕获时有哪些限制。 38. 解释 kernel fusion 在 CUDA 层面的收益来源。 ★39. 什么是 Nsight Systems 与 Nsight Compute？分别用来定位哪类问题。 40. 解释如何用 Nsight 判断一个 kernel 是 compute-bound、memory-bound 还是 latency-bound。

思考与取舍题

42. shared memory 占满不一定比走 L2 快：构造一个"把数据全塞进 shared memory 反而更慢"的场景，并说明此时该看哪个计数器来裁决。
43. warp divergence 消除后 kernel 反而变慢：给出至少两种可能的解释，并说明哪种情况下"消除分支"其实是伪优化。
44. cudaMallocAsync 池化消除了频繁分配开销，却引入碎片的新形态：描述池化后碎片长什么样，以及它与 cudaMalloc 时代碎片在排查方式上的差异。
45. 统一内存的页迁移抖动在高带宽场景反而可能可接受：构造这样一个场景，并给出你判断"可接受"的量化标准。
46. occupancy 冲到 100% 性能反而下降：是资源竞争、缓存污染还是调度开销？设计一组实验把它定位出来。
47. CUDA Graph 捕获后遇到动态 shape 回退到逐 kernel launch：估算高频小 kernel 场景下这次回退的代价量级，并辩论"为 Graph 牺牲 shape 灵活性"值不值。
48. 现象诊断：Nsight 显示 kernel 是 memory-bound，把 occupancy 提高后耗时纹丝不动。候选原因：(a) DRAM 带宽已打满，并发再高也无济于事；(b) L2 命中率下降抵消了收益；(c) kernel 实际卡在延迟等待而非带宽不足。请排序并指出对应的证据。
49. 反直觉追问：循环展开 8 倍导致寄存器溢出到 local memory，性能有时不降反升——什么条件下这是合理的？
50. 取舍辩论：用 maxrregcount 强行压寄存器避免溢出，还是放任编译器自由分配？在"单 kernel 耗时占比高"与"多 kernel 排队"两种上下文里分别投票。
51. 量级估算：一个 kernel 在 108 个 SM 上每周期各读取 128 字节、主频 1.4 GHz：估算它需要的显存带宽并与 HBM 峰值比较；带宽利用率到多少时你才承认"已经打满"？
52. 假设破坏：下一代卡显存带宽翻倍但容量减半：你现有的 kernel 优化清单里，哪些优先级要上调，哪些可以直接删掉？
53. 现象诊断：两个理论访存量相同的 kernel（SoA 与 AoS 布局）实测相差 4 倍。候选原因：(a) 全局内存合并访问粒度差异；(b) L2 sector 利用率不同；(c) warp 发射与调度瓶颈。请排序并给出各自的关键指标。

综合设计题

41. 如果让你写一个 GPU 数据预取或 KV Cache 搬运的 CUDA 组件，你会如何设计线程层次、内存布局、Stream 重叠与错误处理？

工程化与协作面试题库（修订版）

覆盖版本管理、评审测试、CI/CD、文档文化与开源贡献的面试题，共 30 题，标注 ★ 的为面试必会 P0 题。

版本管理与协作流程

1. 解释 Git 的分支模型，以及 trunk-based development 与 Git Flow 的取舍。
2. 什么是 rebase 与 merge？在多人协作的 Infra 项目中如何选择。

代码评审与测试

★3. 解释代码评审的核心关注点，哪些问题应该在评审中拦住，哪些应该留给测试。 ★4. 什么是测试金字塔？单元测试、集成测试、端到端测试在 AI Infra 中如何分配。 5. 解释 mock 与 fake 的区别，在测试 K8s 控制器时如何选择。 ★6. 什么是表驱动测试？Go 中如何用它覆盖调度器或网关的边界条件。 7. 解释竞态检测与压力测试在 CI 中的位置。 8. 什么是 flaky test？在分布式系统测试中如何治理。

CI/CD 与制品管理

9. 解释 CI/CD 流水线的关键阶段，以及如何为 GPU 相关代码设计 CI。
10. 什么是制品仓库？容器镜像与模型文件分别如何做版本管理。
11. 解释语义化版本与 API 兼容性，在 Operator 与 CRD 演进中如何落地。
12. 什么是 feature flag？如何在推理网关中安全地灰度新路由策略。

文档与运行文化

13. 解释设计文档的结构：背景、目标、非目标、方案、取舍、风险、迁移。
14. 什么是 RFC 流程？开源 Infra 项目中 RFC 如何推动架构演进。
15. 解释故障复盘的文化与结构，如何区分根因与触发条件。
16. 什么是 on-call？AI Infra 值班时最常遇到的告警类型有哪些。
17. 解释 runbook 的价值，如何为推理服务与训练任务编写可执行手册。

开源贡献

★18. 什么是开源贡献路径？如何从 good first issue 走到核心模块 patch。 ★19. 解释如何定位一个开源项目的 issue，并提交可被接受的 PR。 20. 解释 DCO sign-off 的含义与 Signed-off-by 的作用，以及一个规范的开源 PR 从分支提交、DCO 签名、CI 通过到 review 礼仪的完整工程流程。 ★21. 一份高质量的 issue 应该包含哪些要素？为什么精确复现步骤、已排查项与环境信息是其中的关键。

思考与取舍题

23. flaky test "重跑通过"为什么是技术债而非解决了问题？辩论一下：容忍 1% 的 flaky 率换取发布速度，这笔账在 AI Infra 项目里应该怎么算。

24. 测试金字塔在没有常驻 GPU 的 CI 里会怎么变形？哪几层会被迫上移或下移，代价转移到了哪里？
25. 被遗忘的 feature flag 会让路由矩阵爆炸：估算 10 个残存 flag、每个 3 个状态的组合空间有多大，并辩论 flag 该不该有强制到期日。
26. RFC 流程过慢会把创新闷死，过快又会让架构失控：设计一条"什么样的改动必须走 RFC"的分界线，并说明它如何保护小改动快速前进。
27. 现象诊断：CI 的偶发失败集中在每天固定时段。候选原因：(a) 共享 GPU runner 被其他团队的定时任务抢占；(b) 依赖镜像的 latest tag 被覆盖；(c) 时段性网络抖动拖慢制品拉取。请排序并给出最小验证步骤。
28. 反直觉追问：评审意见越多代码质量越高吗？当评审延迟迫使开发者绕过流程时，这条正相关的链条会在哪里断掉？
29. 取舍辩论：实验性 notebook 与一次性脚本要不要进主干？为"实验代码进主干的资格"设定一条你自己的红线。
30. 量级估算：一个 5 人团队每周 20 个 PR、每个 PR 需要 2 个 approval、平均评审等待 24 小时：估算这套流程的吞吐上限，并用这个数字辩论"加人、加自动化还是减流程"。

综合设计题

★22. 如果让你把“我做了一个训练数据预取层”讲成面试项目，你会如何组织背景、瓶颈、方案、数据、取舍与反思？

其他补充面试题库（修订版）

覆盖可观测性、SLO 与错误预算、成本与 FinOps、安全与多租户、服务网格与网关生态的面试题，共 57 题，标注 ★ 的为面试必会 P0 题。

可观测性

★1. 解释可观测性三大支柱：metrics、logs、traces 的区别与协同方式。 2. 在 AI Infra 场景中，为什么传统 APM 指标不足以定位 GPU 训练与推理瓶颈？ 3. 什么是 eBPF？它如何在无侵入前提下采集网络、系统调用、I/O 与调度延迟？ 4. 解释 OpenTelemetry 的数据模型，以及它如何统一 trace、metric、log。 5. 解释 OpenTelemetry Collector 的 pipeline 结构（receivers、processors、exporters），以及头部采样与尾部采样策略各适合什么场景？ 6. 什么是持续 profiling？它和采样式 profiling、插桩式 profiling 有何差异？ 7. 解释火焰图在 Go 服务与 GPU 推理服务中的读法差异。 8. 什么是 USE 方法？如何用它对 GPU、NVMe、网络、CPU 做系统级排查？ 9. 什么是 RED 方法？它适合监控哪类服务？ 10. 解释分布式追踪中的上下文传播，以及跨网关、vLLM、存储的 trace 如何串联。 11. 什么是 exemplar？它如何把 metric 异常关联到具体 trace？ 12. 解释日志采样与结构化日志，在高吞吐推理网关中如何控制日志成本？ 13. 什么是审计日志？在 multi-tenant AI 平台中为什么必须与业务日志分离？ 14. 解释指标基数爆炸问题，以及如何为 GPU 多租户场景设计标签。 15. 解释 GPU 的常见故障类型：ECC 错误与 XID 事件分别代表什么？运维上应如何监控与处置？

SLO 与错误预算

★16. 什么是 P50、P95、P99、P999 延迟？为什么 AI 推理服务尤其关注尾延迟？ 17. 解释直方图与摘要的区别，以及 Prometheus 中如何选择。 ★18. 什么是 SLO、SLI、SLA？如何为推理平台定义可执行的 SLO？ 19. 解释错误预算，以及它如何驱动推理服务的发布与回滚策略。

成本与 FinOps

20. 什么是容量规划？如何根据历史 QPS 与 token 吞吐估算 GPU 与存储需求？
21. 解释成本可观测性，如何把 GPU 成本分摊到团队、模型、请求维度？
22. 什么是 FinOps？在 AI Infra 中如何落地 GPU 成本优化闭环？
23. 解释预留实例、竞价实例、Spot GPU 在训练与推理中的取舍。
24. 什么是检查点与抢占恢复？Spot 实例被回收时训练任务如何不丢进度？
25. 解释 token 计量口径：输入、输出与缓存命中折扣分别如何计算？基于这些口径如何设计计费指标？

安全与多租户

★26. 解释推理服务的多租户安全隔离：网络、存储、GPU、KV Cache、日志。 27. 什么是机密计算？它在 AI 推理与训练数据保护中的潜在价值是什么？ 28. 解释模型权重与训练数据的加密存储与传输方案。 29. 什么是供应链安全？容器镜像、模型文件、依赖包分别如何做签名与校验？ 30. 解释 SBOM 在 AI Infra 中的意义，以及如何生成与消费。 31. 什么是策略即代码？如何用 OPA/Gatekeeper 约束 GPU 与存储使用？ 32. 解释 Kubernetes 中的安全上下文与 Pod Security Admission。 33. 什么是 seccomp、AppArmor、SELinux？它们如何限制容器逃逸风险？ 34. 解释 gVisor 与 Kata Containers 的隔离原理与性能代价。 35. 什么是零信任网络？在推理网关与服务网格中如何落地？

服务网格与网关生态

36. 解释 mTLS 在服务网格中的作用，以及它对推理延迟的影响。
37. 什么是 SPIFFE/SPIRE？它如何为工作负载签发身份？ ★38. 解释 API 网关与 Service Mesh 的边界，推理网关该放在哪一层？
38. 什么是 Envoy？它的 filter chain、cluster、listener 模型如何支撑推理网关？
39. 解释 xDS 协议，以及控制面如何动态下发路由与限流策略。
40. 什么是 Wasm 插件？它如何让网关在不重启的情况下扩展鉴权与路由逻辑？

其他

★42. 什么是混沌工程？如何为 AI 训练与推理平台设计故障注入实验？ 43. 解释故障演练中的爆炸半径控制与熔断机制。 44. 什么是 DNS 缓存与解析延迟？大规模集群中如何避免 DNS 成为瓶颈。 45. 解释时间同步的重要性，NTP/PTP 偏差如何影响分布式训练与 tracing。

思考与取舍题

46. 错误预算耗尽即冻结发布，对推理平台可能是错误决策：构造一个"冻结反而放大事故"的场景，并辩论错误预算制度在 AI 平台需要做哪些本地化修改。
47. 指标基数爆炸后，聚合会悄悄丢失长尾：描述一条从"标签太多"到"P999 读数失真"的传导链，并指出链条上最适合拦截的一环。
48. 量级估算：mTLS 在长连接复用与每请求新建连接两种模式下，对 P99 的开销占比分别能到多少？在什么连接模式下它才"小到可以忽略"？
49. 把 GPU 成本摊到每个请求会诱导哪些博弈行为？至少列出三种，并说明你会在计量设计上优先堵住哪个口子。
50. 混沌注入打到共享存储后爆炸半径失控：描述一次"演练变成事故"的传导路径，并辩论共享存储该不该被排除在演练对象之外。
51. 现象诊断：新租户接入后全平台 P99 上升，但每个租户自己的指标都正常。候选原因：(a) 聚合指标被新租户的负载模式拉偏；(b) KV Cache 与计算资源发生争抢；(c) 监控采样偏差制造了假象。请排序并给出验证路径。
52. 全量链路追踪上线后 P99 反而涨了 5 ms：开销来自哪里？什么条件下这个代价值得付，什么条件下应该退回采样？
53. 取舍辩论："把 SLO 从 P99=100 ms 放宽到 200 ms 能省 30% GPU 成本"——这项决策应该由谁、基于什么数据拍板？为反对方写出最强的一条理由。
54. 假设破坏：假设所有监控数据丢失了 10 分钟，事后你能靠什么判断这 10 分钟内服务是否出过问题？你的系统里还剩几条独立证据链？
55. 量级估算：一个只有 20 个桶、覆盖 1 ms–10 s 的直方图，读 P999 时最大误差能有多大？桶宽应该向哪一端倾斜？
56. 现象诊断：训练任务的 XID 错误集中在每个 epoch 的特定 step。候选原因：(a) 该 step 的显存峰值触发 OOM 类错误；(b) 集合通信在特定步的拓扑路径上出问题；(c) 功率墙降频后触发看门狗超时。请排序并说明各自的取证手段。
57. 假设破坏："NTP 偏差几十毫秒无所谓"——给出两个会被这句话打脸的场景，并排序哪一个会先暴露。

15-项目考察（v3 扩充版 · 150 题）

> 2026-10-05 v3：对齐 04/05/06/07 v3，从 56 题扩到 **150 题**。项目深挖占技术面试时长的一半以上，每题答案必须来自你自己的项目证据（对应 `04/05` 的闸门、旗舰图与"诚实清单"）。带 ★ 的题必须能脱稿讲 5 分钟以上；带 ⚠ 的是"旧设计已被推翻"的题，专门用来考察你能不能讲清自己如何纠错。
>
> 分布：A 通用 18 · B P1 TrainCache 43 · C P2 KVRouter 35 · D 小项目与微弹药 26 · E 开源贡献 8 · F 收尾战术 3 · G 追问链 17。

A. 通用项目考察框架（1–18，任何项目都会被问到）

1. ★ 用 STAR 结构介绍你最有挑战的项目：背景、瓶颈、方案、结果、取舍。

2. ★ 你项目的 baseline 是什么？怎么测的？硬件、软件版本（镜像 digest/chart 版本）、数据集规模分别是多少？
3. 项目里最难的 bug 是什么？从发现到定位到修复的完整过程，哪一步耗时最久？
4. 项目里最难的设计决策是什么？被否决的备选方案有哪些，为什么否决？
5. 你的性能数据怎么打点？如何保证可信（对照组、重复次数、方差/置信区间、同一环境）？
6. ★ 如果并发/数据规模再上去一到两个数量级，你的系统会先崩在哪？为什么？
7. 如果重做一遍，你会改掉什么、保留什么？（要求能说出 v1→v3 真实演进中的至少一次自我否定）
8. 这个项目为什么不用现成的 X（Fluid / JuiceFS / AIBrix / Production Stack）？差距和边界分别是什么？
9. 项目里哪部分是借鉴现有方案、哪部分是自己的增量？借鉴部分你读到什么程度（代码行级还是文档级）？
10. 项目代码量多少？测试怎么分层（单测/e2e/混沌）？本地与 CI 怎么跑？
11. ★ 讲一个你的"阴性结论"：假设是什么、怎么预注册的、数据如何把它推翻、你如何呈现它——阴性结果为什么反而是高价值证据？
12. 你的竞品现勘方法论是什么：读哪些材料、抓什么工件（webhook 注入的 Pod spec / PR / 事件流）、如何防止"我以为它没有"变成假缺口？
13. 如果现勘结论推翻了你的立项前提（上游已经做了），你的止损动作是什么？
14. 项目时间盒是怎么管理的？砍掉过什么、按什么优先级砍的？砍掉的东西如何向面试官交代？
15. 你项目里 AI 工具参与了哪些部分？哪些是你绝对不让 AI 碰的（事实、测量、commit 背后的理解）？
16. 你怎么让别人一键复现你的核心实验（脚本、锁定版本、种子、tc 参数清单）？
17. 这个系统最大的正确性风险（不是性能风险）是什么？你用什么机制兜底？
18. 作为个人项目，被问"团队协作体现在哪"时，你如何用开源 review/issue 攻防来回答？

B. P1 TrainCache v3：预热-调度闭环（19–61）

B1 定位、现勘与实验方法（19–26）

19. ★ Fluid + JuiceFSRuntime 已经能做数据亲和调度，你的缝隙到底在哪？请出示现勘证据：它的 webhook 注入的是什么（Pod 创建时一次的粗粒度 nodeAffinity）、DataLoad 与调度决策为什么不闭环。
20. ★ 五组对照组分别回答什么问题？为什么必须和**真实 Fluid**而不是你想象的 Fluid 比？如果你的方案在主区间跑输 Fluid，你怎么办？
21. ⚠ 早期版本把项目说成"数据层 CDN/内容寻址"，v3 为什么改为"调度驱动的预热闭环"？讲清你判断失误的来龙去脉。
22. 现勘闸门要求"抓 webhook 注入的 Pod spec、记录 DataLoad 与 Pod 时序"——具体怎么抓？如果证实 Fluid 已有块级进度门控，叙事如何收缩？
23. ★ 头号指标为什么是 Job 提交→首个 step、epoch makespan、dataloader stall 占比，而不是"门控等待时间"或缓存命中率？预热字节×fanout、源站出口这些成本指标为什么必须一起报？
24. ★ 讲旗舰图：源站带宽 × 缓存欠配 × fanout 的盈亏曲面是怎么设计扫点的？哪些区间你预期门控必亏？假设是怎么预注册的？

25. 单机单物理盘上你如何造出真实冷热差（tc tbf/netem 只作用在 Agent→MinIO 路径、命中本地盘不过源站）？为什么这个装置是可信的、它的外推边界在哪？
26. 三轨负载（百万小文件 / webdataset 或 MDS 百 MB 分片 / 多 Job 倾斜 trace）各代表什么历史与现实？为什么小文件轨不能当主臂？每轮实验为什么要 `sync; echo 3 > drop_caches`、重复 ≥7 次并报 bootstrap CI？

B2 双层标识与缓存存储层（27–32）

27. ★ 画一遍"索引层 `(path, chunkIndex)` 路径寻址 + 存储层 xxh3-128 CAS"的双层模型。⚠ 为什么旧文档把 `xxh3(path+index)` 叫"内容寻址"是术语硬伤？内容哈希从哪个时刻才存在？
28. xxh3-128 相比 SHA-256 省什么、赌什么？128 位在你数据规模下的碰撞概率是多少？什么信任边界下必须换密码学哈希？
29. CAS + refcount 如何让"内置校验和相同的不同分片"安全去重？哈希只在下载流水里计算，如何避免额外一遍读盘？
30. ★ tmp + rename 的原子可见性解决什么具体故障（训练侧读到半块）？rename 崩溃、目录已存在、跨文件系统 rename 各自的行为是什么？
31. 缓存块为什么可以不逐块 fsync？这个假设依赖什么（崩溃后回源可重取 + CAS 校验）？什么场景会打破它？
32. 双水位驱逐（85%→75%）为什么要滞回而不是单水位？refcount 如何钉住在读块？磁盘配额被其他进程抢占时怎么办？

B3 Agent 与控制面工程（33–37）

33. workqueue + errgroup + 按字节加权信号量（bytes-in-flight 512MiB）三者各解决什么问题？为什么信号量按字节而不是按请求数？
34. Agent 重启后如何重建状态（扫描数据盘→校验 manifest→重建索引）？扫描期间请求如何处理？
35. 缓存命中率的统计口径是什么？旁路采样（inotify/日志）有哪些误差来源，你如何与 eBPF/`/proc/diskstats` 口径交叉验证？
36. ★ 百万块清单为什么不能进 CRD status（etcd 对象 ~1.5MB 上限）？index shard 放对象存储、status 只留 readyRatio/位图摘要的方案，在分片更新竞争和 controller 重启下如何保持正确？
37. manifest 与子集订阅如何用增量 List + ETag/版本号避免全量拉取？Job 声明消费子集的语义错配（声明了不存在的 shard）如何暴露成事件而不是静默卡住？

B4 核心 1：预热-调度闭环（38–47）

38. ★ 白板讲全回路：Job 声明子集 → controller 向候选节点下发块清单 → Agent 预热上报 readyRatio → gate 摘除/打分 → Job 运行中状态如何反馈。哪一步是控制回路的"测量点"、哪步是"执行器"？
39. ★ SchedulingGate（K8s ≥1.26）的摘除如何幂等？controller 重启、重复 reconcile、gate 已被人工摘除三种情况分别怎样？为什么不会重复摘或漏摘？
40. ★ Score 插件为什么严禁网络调用？它只读 informer 本地缓存，缓存 TTL 内的陈旧信息会造成什么错误调度？错误的代价与一次 RTT 的代价怎么权衡？

41. required 与 preferred 两策略的语义差别？总超时（默认 5 分钟）后降级到"去 gate/去亲和正常调度"——为什么"Job 永不无限 Pending"是第一正确性原则？
42. 多副本 fanout 放置策略（2–3 副本）如何决定？预热字节随 fanout 线性涨、命中收益边际递减，这个 Pareto 权衡你的数据显示最优点在哪？
43. 预热带宽预算如何限制，防止把在线任务的源站带宽吃光？被挤压时优先保谁（在途 Job 预热 vs 新 Job 预热）？
44. ★ 什么条件下门控**反而比直接调度慢**（源站极快、预热欠配、fanout=1）？你的曲面阴性区间里这些条件占多大比例？
45. 为什么要求 K8s ≥1.26？在不支持 scheduling gate 的集群上你的降级方案是什么？kind 里如何 e2e 验证 gate 行为？
46. 这个 Score 插件与 S6 拓扑打分插件同时启用时，框架如何归一化两个插件的分数？数据 readyRatio 与 NUMA 拓扑冲突时谁让谁？
47. 如果核心 1 被现勘证伪（Fluid 新本版已闭环），你交付的最小价值是什么（复现数据 + 参数量化 + 上游 issue）？

B5 核心 2：准入与预取（48–52）

48. ★ 为什么 W-TinyLFU 在顺序 epoch 全量读取上预期与 LRU/LFU 无差异？这个阴性假设是怎么设计实验确认的？
49. count-min sketch 与 doorkeeper 的参数（sketch 宽度/深度、采样频率）怎么定？参数敏感性怎么扫？
50. 多 Job 倾斜 trace（缓存欠配 30%、A 反复读 10% 热分片、B 全量扫描）如何构造才真实？确定性 shuffle 种子如何管理？
51. 自适应预取窗口（类 TCP 拥塞控制）的降窗/增窗触发信号是什么？为什么用 stall 信号而不是命中率？
52. 如果准入策略最终全线无差异，你在简历和面试里怎么讲这个结论，才不显得项目失败？

B6 支撑臂与正确性（53–58）

53. 同一分片多 Job 并发回源如何用 singleflight 合并？流式扇出（io.Pipe）相比"先落盘再分发"省什么、引入什么失败模式？
54. AIMD 式背压的参数如何选？Agent 崩溃时已收到部分数据的下游怎么处理？
55. ★ ⚠ checkpoint 的旧错误（权重到齐就让训练 step）为什么会改变收敛轨迹？optimizer state（momentum/Adam 一阶二阶矩）缺失后数学上发生了什么？推理/eval 为什么可以权重先到先用？
56. sidecar checkpoint 上传的 multipart + manifest 原子发布点在哪？Range GET 并发恢复如何校验分片完整性？加载与初始化重叠具体重叠了哪两段时间？
57. P2P 臂为什么定位为"在 JuiceFS CE 上复现 EE 分布式缓存组语义"？一致性哈希环、peer 回源、失败穿透、origin 保底 20% 各解决什么问题？为什么不把这些当自研卖点？
58. 混沌测试四件套（Agent kill -9 / netem 分区 / 磁盘满 / 位翻转）各自的恢复判据是什么？CAS 如何把位翻转变成可检测错误而不是静默脏数据？RTO 怎么测？

B7 RDMA 实验臂（59–61）

59. 没有 RDMA 网卡你怎么做真实验证（Soft-RoCE `rdma_rxe` + 真 libibverbs 代码）？为什么功能可验证、性能数字不可采信？
60. ★ MR 注册缓存与 4MiB 块大小的关系？为什么建议 C++ 独立传输组件 + 薄 C ABI，而不是 cgo 直接把 libibverbs 链进 Go 热进程（线程模型、信号、故障域）？
61. 云真值（阿里云 eRDMA 按小时租）与本机 Soft-RoCE 结果如何对照？租卡窗口的实验清单怎么提前排，才能把每小时花在刀刃上？

C. P2 KVRouter v3：复现 • 测量 • 前沿缺口（62–96）

C1 定位、生态与现勘闸门（62–66）

62. ★ 2026 年 AIBrix（v0.7+）、llm-d、Production Stack 已经覆盖了哪些能力（软评分路由、KV 事件消费、KV offload、多副本 HA、P/D 分离）？你凭什么还做这个项目？
63. ★ 讲清项目的三重身份：工业方案的独立复现者、双模式机制的量化测量者、"冷启动协同"缺口的上游贡献者。为什么这是比"自研更强网关"更可信的本科生叙事？
64. ★ 阶段 0 头号风险：vLLM **CPU 后端是否上报与 GPU 后端相同 schema 的 KV 事件**？如果不支持，精准模式如何用符合 schema 的 mock 事件流开发、真值又靠什么窗口补齐？
65. 阿里云文档区分的"精准模式/估算模式"各自信息源是什么？AIBrix 消费 KV 事件的实现在哪几个 PR/目录，你读到什么程度？
66. 如果你现勘发现 AIBrix 已经实现了冷启动预热/放量协同，候选 A/B/C 如何切换（转增量池、换缺口、收缩为复现）？

C2 基座：SSE 反代与负载台（67–72）

67. ★ Go SSE 反代你踩过的五个坑：flush 时机（Corking/Flusher）、客户端断开如何用 context 传播到上游、gzip 与 SSE 的冲突、idle 超时、上游连接复用。逐条讲验证方法。
68. 流式转发中"上游还在吐、客户端已断"时，如果不取消上游会浪费什么（GPU 算力/KV 位）？你怎么观测这种浪费？
69. 为什么不能简单 `httputil.ReverseProxy` 一把梭：你需要在流上观察/计量什么（首 token、流式 token 数），怎么做到不缓冲全响应？
70. MetricsView 抽象解决什么问题？不同后端（vLLM/AIBrix mock）指标口径漂移如何统一？
71. ★ 参数化负载发生器为什么是一级公民？它有哪些维度（前缀共享率、长短请求混合、到达分布、think time）？参数如何锁定成可复现实验？
72. 压测客户端自己饱和会怎样污染数据？你如何用独立哨兵请求、CPU 监控和多机发压证明客户端不是瓶颈？

C3 核心 1：双模式 KV 索引（73–80）

73. ★ 估算模式（radix 树 + 路由历史 + TTL）为什么本质不准——哪些事件它永远看不见（外部路由、淘汰、被动驱逐）？既然不准为什么还要**亲手复现**而不是只读代码？

74. ★ 精准模式：消费 vLLM KV Cache 事件后，你维护什么状态机（stored/evicted 事件、块递归哈希、token 块→副本倒排索引）？事件乱序/重复/丢事件分别怎么处理？
75. 块递归哈希（`h_i = H(h_{i-1}, block_i)`）为什么能让任意前缀的缓存状态变成可定位的 key？block size（如 256 token）对粒度与事件量的权衡？
76. 网关不持有 tokenizer 时怎么获得 token 边界（三方案：LLM 返回 usage/外置 tokenizer/消息边界近似）？⚠ 旧版"消息边界哈希"为什么被降级淘汰，它的误差朝哪个不安全方向？
77. 双倒排索引（token 块→副本、副本→块集合）如何支撑路由查询与副本摘除？Pod UID 变化（重建）时为什么必须按 UID 而不是 endpoint 摘除？
78. 索引滞后于真实缓存状态时，路由到"以为有缓存"的副本会怎样？你如何打点量化失配率，失配后回退路径是什么？
79. ★ 讲旗舰图 1：自研估算、自研精准、真实 AIBrix 三条曲线在前缀共享率 × 缓存欠配二维平面上怎么测？三方公平对照要锁住哪些变量？
80. 如果精准模式只在极小前缀区间赢估算模式，这个结论对工业界的含义是什么？

C4 核心 2：缓存感知弹性协同（81–88）

81. ★ 白板讲冷启动饿死机制：新副本缓存全冷，软评分路由器为什么反而可能在一段时间内不给它流量或骤然全量放量？TTFT 尖峰与命中率凹陷的因果链是什么？
82. ★ 事件研究（event study）方法：以伸缩事件为 t=0 对齐多个指标（TTFT、命中率、放量比例），如何控制"扩容本身发生在流量高峰"的内生性偏差？
83. 热前缀预热请求如何构造才不污染真实请求队列与指标？预热多少前缀、从哪取前缀分布（trace top-k vs 在线学习）？
84. 带迟滞的加权放量曲线（缓慢升权 + 命中率反馈）参数怎么定？固定斜率与按命中率自适应两种方案你怎么对照？
85. 缩容 drain 如何按"缓存价值损失"选 victim？正在处理的长请求、租约期、优雅终止超时如何协同？
86. Little 定律排队模型在冷启动问题里是主角还是配角？为什么不用 ML 预测放量时机？
87. ★ 讲旗舰图 2：伸缩事件时间轴上的 TTFT 尖峰/命中率凹陷，baseline（无协同）与你的策略对比，置信区间如何报告？
88. 回馈 AIBrix 为什么必须 issue/discussion 先行？如果维护者说"我们已有 WIP"，你如何把被指路变成收益而不是失败？

C5 增量池与只调研项（89–92）

89. 增量池为什么只允许选 1 个？默认选项"流式故障安全重试"的幂等键、generation token、非幂等生成请求如何处理？
90. 多租户缓存干扰实验想回答什么问题？为什么它必须在冷启动协同之后而不是并行？
91. ★ P/D 分离、NIXL/SHFS/Mooncake 的 KV 传输为什么"只调研不构建"？能白板讲清 prefill/decode 资源画像差异与 KV 传输在中间的角色吗？
92. Registry HA 为什么只写决策分析？AIBrix 已有 Redis 同步方案的情况下，你自建一致性键值的边际价值为什么是负的？LMCache 的一次集成实验边界是什么？

C6 实验、硬件与计量（93–96）

93. CPU 模式下哪些实验可信（机制正确性、索引逻辑、mock 事件下的路由）、哪些绝对不能报性能数字（注意力延迟、真实吞吐）？
94. 租 GPU 真值窗口的校准实验怎么排（最小模型、固定镜像 digest、按小时预算内的扫点顺序）？
95. ≥7 次重复、bootstrap 95% CI、固定随机种子与请求轨迹——在"两个 vLLM 副本"这种有内部调度非确定性的系统里，哪些种子锁得住、哪些锁不住，你怎么处理锁不住的部分？
96. token 计量的口径：input/output/缓存命中折扣怎么定义才不和厂商口径冲突？SSE 流上如何精确数 output token？

D. 小项目与微弹药（97–122）

S1 mini-Raft（97–101）

97. ★ votedFor 不落盘如何在一次分区恢复后造成同任期双投？写出时序。
98. leader 为什么不能直接提交前任任期的日志（§5.4.2）？写出一个只有旧任期日志时的反例时序。
99. ★ 你的无主停机窗口实验怎么设计：kill -9 leader ≥100 次、窗口如何定义（kill 时刻→新 leader 首个成功写）、中位数/P99 是多少、多数派/少数派/不对称分区三种场景差异？
100. ★ 《toy raft 与 etcd raft 的 5 个差异》挑两个讲：Ready 批量化通道、unstable log/Storage 接口抽象分别解决了 toy 实现的什么痛点？
101. Pre-Vote 防的是什么具体故障（分区节点持旧 term 归队扰乱选举）？joint consensus 两阶段成员变更为什么不能一步切？白板讲，不实现也要会。

S6 拓扑感知调度插件（102–107）

102. ★ 白板讲全链路：device plugin TopologyInfo → kubelet Topology Manager → NFD topology-updater → NodeResourceTopology CRD → 调度插件 Filter/Score → bind。
103. 官方 `NodeResourceTopologyMatch`/ResourceAllocationScore 已经存在，你的项目为什么不是"发明"？你的复现/增量姿势是什么？
104. 自研插件的骨架：`New(obj, handle)` 里读什么、PreFilter 与 Score 各自的时序与返回值语义、NormalizeScore 什么时候必须实现？
105. 配 KubeSchedulerConfiguration v1 踩过哪些坑（apiVersion、多 profile、schedulerName、pluginConfig 位置）？自定义 scheduler 的 RBAC 最少要哪些权限？
106. ★ 单 NUMA 游戏本上用 annotation/fake NRT 拓扑做实验为什么是正当的（官方 e2e 也 mock）？你如何诚实标注"没验证真机绑核性能"？
107. ★ 数据亲和 × 拓扑亲和的权重-决策曲线怎么画？多插件分数归一化为什么没有银弹？DRA 相对 device plugin 路线改变了什么？

S4 bitcask（108–110）

108. fsync 三档（逐条/group commit/OS 刷盘）的吞吐与崩溃窗口数字分别是多少？group commit 的 fsync 协调者崩溃时边界行为是什么？

09. merge 的 manifest 原子切换：rename 数据 → rename hint → 写 manifest，崩溃落在每一步分别如何恢复？⚠ 记录头时间戳为什么用 8 字节（2038）？
10. 10M key 实测的索引内存/数据量比是多少？数据量超内存后 bitcask 为什么输给 LSM（SSTable+稀疏索引+分层 compaction 的读写放大对比）？ merge 期间写延迟抖动的丑数字敢报吗？

S3 eBPF（111–112）

11. block I/O 延迟直方图为什么用 tracepoint 配对（issue/complete 按 request 指针）+ per-CPU map 内核侧聚合？ringbuf 方案在高 IOPS 下会怎样？
12. ★ 为什么 preheat 命中 page cache 时没有 block I/O 事件？readahead 产生的 major fault 为什么可能不记在消费者进程名下——这个归因陷阱如何验证？

S2 io_uring 臂（113–114，随 P1）

13. O_DIRECT 三重对齐与 overlayfs EINVAL 的根因？你最后落盘在什么文件系统上验证？
14. ★ 4MiB 大块 HTTP 抓取为什么是网络 bound、io_uring 省 syscall 对吞吐预期中性？你的实验是否验证了这个阴性预期？小文件高 IOPS 臂的数字才是它的战场——两边数字都要报。

M1 mini-container（115–117）

15. ★ 你用了哪几个 namespace（NEWPID/UTS/NS/NET/USER）？ cgroup v2 下 `cpu.max` / `memory.max` 写在哪个层级，进程怎么加入（ `cgroup.procs` ）？ v2 统一层级解决了 v1 什么问题？
16. pivot_root 与 chroot 的本质差别？为什么 rootless 容器必须依赖 user namespace 的 uid 映射？
17. 容器为什么不是虚拟机（共享内核意味着什么攻击面与限制）？ kubelet → CRI → containerd → runC 的调用链里你写的那 200 行对应哪一层？

M2 mini-CNI（118–119）

18. CNI 插件在容器创建的哪个时刻被调用（netns 已建、rootfs 之后）？ ADD/DEL/CHECK 从哪里读配置与 netns 路径？你的 IPAM 地址池如何防重启后地址泄漏？
19. 你的 bridge 方案为什么只解决同节点？跨节点 VXLAN/路由/BGP、以及 Service 的 iptables/IPVS 为什么是独立两层——能讲清你没做的部分边界吗？

M3 mini-CSI（120–122）

20. ★ kubelet plugin registration 握手全过程：plugins_registry 下的 socket、NotifyRegistrationStatus 调用、socket 生命周期；kind 里注册不上你怎么用 `csc` 直接调 gRPC 降级验证？
21. CSI 的 Attach（ControllerPublish）→ Stage（NodeStage）→ Publish（NodePublish）三段各自为什么存在？为什么你的 hostPath bind mount 最小实现只需要 Publish？
22. ★ Fluid/JuiceFS 为什么必须自带 CSI driver——FUSE 挂载如何经 CSI 注入 Pod 的 mount namespace？这个微弹药在你 P1 现勘 Fluid CSI 时给了你什么别人没有的视角？

E. 开源贡献考察（123–130）

23. ★ 讲一个你的完整贡献故事：issue 怎么从现勘数据里长出来、根因与证据、方案分歧、maintainer 反对什么、最终形态、未被接受的部分。
24. 为什么"同一社区 docs/test→code 的信任线"比"横跨多个社区各一个 typo PR"可信？scheduler-plugins 上的练习 PR 你如何定位？
25. DCO 与 CLA 的法律含义区别是什么？JuiceFS 要求签 CLA、Fluid 只要 DCO——首 PR 时各自的机器人流程是什么，漏签如何补救？
26. ★ 为什么 feature 类必须 issue/discussion 先行？讲一个"被维护者指向已有 WIP"或"PR 被关闭"的真实经历，你如何体面退场并保留信用？
27. AIBrix 维护者分工是什么（routing/autoscaling/KV offload/benchmark 各找谁）？为什么乱 @ 维护者是减分项？
28. AIBrix 贡献文档为什么要求 review 期间避免不必要的 force-push？48 小时响应、每次 push 留言说明改动——这些礼仪背后 reviewer 的真实成本是什么？
29. 简历上如果只有一个用来走流程的 docs PR，你如何诚实表述而不包装成"活跃开源贡献者"？
30. 如果上游需求和你的项目方向冲突，你怎么取舍？为什么原则必须是"项目产生贡献，而不是贡献牵引项目"？

F. 面试收尾战术（131–133）

31. 被问到不会的问题的标准应对：承认边界（不装）+ 给出排查路径 + 关联你确实懂的领域。现场示范一次。
32. "你有什么想问我们的"：三个问题——团队当前最痛的技术挑战 / 推理或数据基建现状 / 实习生将拥有的真实项目与 mentor 机制。
33. 被压力面否定（"你这就是玩具/没价值/别人早做了"）时，如何不辩解、用证据和边界感回应？

G. 追问链训练（134–150，"防御的防御"）

> 用法：不是独立题。每答完一个项目问题，让追问者沿这里再钻两层。答不利索说明对应增量没吃透，回 04/05 重看。

34. ★ 你说吞吐/makespan 改善了 X%——误差范围多少、几轮、换机还成立吗、baseline 是同一时刻同一环境测的吗？
35. ★ "这不就是 Fluid / AIBrix 已经做过的吗"——三段式回应：承认共性 → 指出你的差异点与证据 → 给出同场对照数据。
36. 你的数字被质疑"样本太小/CI 太宽"——补实验的具体方案是什么（加轮次、配对设计、分层 bootstrap）？
37. 这个设计放大 100 倍（节点数/块数/QPS/模型副本）第一个失效的组件是什么？修复方向？
38. "为什么不用更简单的方案 X？"——先说出 X 的三个优点，再用你的数据说明它在哪个条件区间输。

39. 你修过的那个最难 bug，如果当时没发现，会在生产中以什么告警/什么用户症状爆发？
40. 你方案里最"丑"的部分是什么（mock 事件/单机 tc 装置/手工拓扑）？为什么容忍它、什么条件下必须还技术债？
41. 代码量最大的模块有多少是 AI 生成的？maintainer 问"这行为什么这么写"时你如何证明每一行你都懂？
42. 如果团队要求换语言栈（Java/Python）或换调度生态，你的设计哪些能活、哪些要重写？
43. 你引用的论文（AIBrix paper / Fluid TPDS / PagedAttention）实验环境与你差多远？那些差异如何影响你借用的结论？
44. 如果 mentor/评审人否定你的方案，讲一次真实经历：你如何用数据回应、如何区分"我被说服了"和"我只是服从权威"？
45. AI Infra 平台组与存储内核组两个 offer，基于 P1/P2/S6/M3 的具体经历，你的选择框架是什么？
46. ★ "做了半天结果是阴性的，这项目有什么用？"——你如何把预注册假设、测量纪律和对工业界的"省坑"价值讲成加分项？
47. "一个本科生在游戏本上复现工业系统，结论凭什么让我信？"——讲清哪些结论不依赖硬件规模（机制、失配、时序）、哪些必须云真值，以及你如何主动标注这两类。
48. 这么多项目并行，你的真实时间分配是什么？哪个里程碑延期过、为什么、如何调整的？（考真实性，不许报完美流水账）
49. 如果到 2027.2 投递时目标 PR 还没 merge，你的简历与叙事如何调整（在 review 的证据链/issue 数据/项目本体）？
50. ★ 讲一个你自己判断错误、后来亲手推翻的设计决策（提示：内容寻址误称、checkpoint 语义、P2 自研网关任选）——你是被什么证据说服的？纠错过程比正确答案更重要。

使用方法：A 类先练到脱口而出；B/C 每题对应 04/05 一个设计点，每过一个闸门/完成一张旗舰图回来重答；D 类随小项目与微弹药完成补答案；E 类在每次真实 issue/PR 后更新；G 类用于模拟面试钻两层。所有答案必须来自真实数据与真实过程，禁止编造数字；⚠ 题不回避，它们是"会自我纠错"最有力的证据。